

无线电干扰源的快速自动定位与清除

摘 要

无线电干扰源的定位与清除是无线电频谱管理的重要任务。本文针对无线电干扰源的快速自动定位与清除问题，围绕圆形监测区域内干扰源交会定位、检测点优选、机器狗自主搜索清除展开建模研究。为无线电干扰源的快速自动定位与清除提供具体方案。

针对问题一，首先，本题核心是交会定位法下定位多边形直径的计算以及外接圆覆盖性判断。由附录可知，每个检测点得到的示向度存在 $\pm 1^\circ$ 的固定误差，每个检测点对应两条边界射线；两个检测点的两组射线相交，形成四边形（定位多边形），该多边形就是干扰源可能存在的几何区域。然后，设计算法，输入若干检测点坐标与对应示向度，生成每一个示向度对应的 $\pm 1^\circ$ 边界射线，求全部射线的相交包围多边形（交会定位区域），再计算多边形直径，即多边形所有顶点两两之间距离最大值。最后验证，以该多边形直径作为直径作圆，判断该圆是否一定能够完整包含整个定位多边形。

针对问题二，给定第一个检测点位置和该点测得的干扰源示向度，要选择第二个检测点，使得交会之后定位区域尽可能小，也就是定位精度最优。已知 S_1 位置、示向度 θ ，在平面上寻找 S_2 ，使得 S_1, G, S_2 交会夹角尽量靠近 90° ，同时 S_2 处于干扰源有效接收范围内。据此确定第二个检测点的候选区域，排除会造成交会夹角过小的位置。需要区分禁止区域（两射线接近平行的扇形区域）与推荐候选区域，为后续机器狗点位选择提供理论依据。

针对问题三，这是个全干扰源场景问题。首先，采用分区扫描，螺旋遍历等路径方案，在路径关键点循环扫描 1-20 全部频道。一旦某个频道检测得到示向度，调用问题 1、2 的交会定位，增设第二个合适检测点，得到干扰源的估计位置。然后，机器狗向估计干扰源位置移动，接近至 20m 以内，执行光学定位清除。最后，保证全部干扰源尽可能被清除，总时间尽可能短。经模拟器三次正式测试，清除干扰源个数分别为 16、10、15，平均定位清除时间 232.65 s、328.02 s、238.65 s，程序运行时间均在 3.2 s 以内，全部干扰源均被有效清除，无未解决频道。

针对问题四，这是个混合场景问题，在问题三的基础上增加定向干扰源。首先，对同一频道，在空间多个不同方位的检测点重复检测，降低落入定向源盲区的概率。然后，优化路径规划，对疑似存在干扰源的方向多角度布点测向。修改交会定位逻辑，要考虑“无信号”结果也蕴含信息：说明检测点位于定向干扰源辐射扇形之外。最后，经模拟器三次正式测试，清除干扰源个数分别为 10、13、16，平均定位清除时间 558.51 s、480.37 s、358.99 s，程序运行时间均在 7.6 s 以内，无未解决频道，验证了策略对两类干扰源混合场景的有效性与鲁棒性。

关键词：交会定位；示向度；机器狗路径规划；无线电干扰源

一 问题重述

1.1 问题背景

无线电干扰源定位清除是无线电频谱管理的重要工作，干扰源定位分为初步监测、区域性排查、精准定位三个阶段。传统方案依靠人工携带测向机徒步完成精准定位，工作效率有限且存在安全风险。现需要研发搭载于机器狗的自动定位清除系统，替代人工完成干扰源的定位与清除作业，以提高定位及清除效率。

1.2 问题要求

在已经确定干扰源处于半径 1800 米圆形目标区域的前提下，以圆形区域中心为坐标原点，正东为 x 轴正方向，正北为 y 轴正方向，结合测向机示向度交会定位原理、机器狗运动与设备工作时间约束，完成以下四项问题研究。

问题一：给定若干检测点坐标，以及同一干扰源在各检测点对应的示向度，基于交会定位法设计算法，计算交会得到的定位多边形区域的直径，同时判断以该直径为直径所作的圆是否可以完全覆盖该定位区域。

问题二：要求在已知全向干扰源于第一个检测点 S_1 测得示向度的条件下确定第二个检测点 S_2 的选择策略与候选区域以保障交会定位效果，由于全向干扰源可在有效接收半径 1000-1500 米范围内任意方位被检测，测向存在 $\pm 1^\circ$ 角度误差，干扰源真实位置仅能确定在 S_1 出发的示向度误差角域内，定位效果取决于交会夹角，夹角越接近 90° 定位多边形越紧凑，夹角过小或接近 180° 会造成定位区域狭长、定位精度恶化，因此选择第二个检测点时应优先保证潜在干扰源处交会夹角趋近 90° ，避开会产生过小或过大交会夹角的共线扇形区域，同时要满足信号接收条件，保证 S_2 处于干扰源有效接收覆盖范围内，兼顾两点不宜过近导致测向区分度不足，还要处于半径 1800 米的目标圆形区域内，并优先选取移动距离短的点位降低机器狗的时间开销，综合方位约束、距离上下限约束以及目标区域边界约束，最终第二个检测点的候选区域为垂直于示向度方向两侧、剔除共线危险扇形、受距离阈值限制且不超出目标大圆范围的扇形 - 环形复合平面区域。

问题三：目标区域内仅存在全向干扰源，干扰源总数量为 10-16 个，具体数目、位置、频道均未知。机器狗从坐标原点出发，初始频道为 1，在半径 1800 米圆形区域内完成全部干扰源定位清除任务，要求尽可能缩短任务耗时。设计机器狗自动搜索、定位与清除的整体策略与算法，利用模拟器开展演练测试，采用干扰源清除比例、平均定位清除时间作为评价指标，完成三次正式测试，按照规定表格记录测试结果，并保存模拟器输出的日志文件。任务受虚拟测试窗口 25 分钟、程序最长运行 20 分钟双重时间限制。

问题四：目标区域同时存在全向干扰源和定向干扰源，总个数依旧为 10-16 个，定向干扰源的数量、发射方向均未知，其余条件、时间约束与问题 3 保持一致。考虑定向干扰源存在信号辐射盲区的特点，改进检测、定位与清除策略及对应算法。通过模拟器

演练验证算法有效性，完成三次正式测试，输出测试结果表格并留存测试日志。

二 问题分析

本题围绕机器狗平台下无线电干扰源快速自动定位清除任务展开，分为几何定位理论、检测点优选、单类干扰源搜索清除、混合干扰源搜索清除四个子问题。已知干扰源分布在半径 1800m 圆形区域内，测向存在 $\pm 1^\circ$ 角度误差，机器狗的移动、频道切换、测向检测、光学清除均存在固定时间开销，需要结合交会定位几何原理，兼顾定位精度、搜索路径、时间开销，设计算法与自动化执行策略，并依托模拟器完成测试验证。

2.1 问题一分析

对于问题一，本问题属于平面几何计算问题，不涉及机器狗移动控制。测向机对同一干扰源在各个检测点获得示向度，受仪器与环境影响存在 $\pm 1^\circ$ 误差，每个检测点会生成左右两条边界射线；多个检测点的全部边界射线相交，围成凸多边形，即为交会定位区域，代表干扰源所有可能的位置集合。

需要解决两点核心任务：第一，设计算法，输入检测点坐标与对应示向度，生成全部误差边界射线，求取射线相交构成的定位多边形，再求解多边形直径，即区域内任意两点距离的最大值。第二，分析以该直径为直径作圆，能否完全覆盖定位多边形。

根据凸集合几何性质，凸多边形的直径是顶点之间的最大距离。若定位区域为凸多边形，以多边形直径为直径的圆必然可以覆盖整个凸多边形；若射线交会出现凹区域，则需要重新讨论。本问题是后面 2-4 问定位计算的基础，定位多边形的大小直接反映定位误差，是评价定位效果的核心指标。

2.2 问题二分析

在已经获得第一个检测点的示向度前提下，要选择第二个检测点，使交会之后的定位区域尽可能小，实现更好定位精度。交会定位的精度高度依赖两个检测点相对于干扰源的交会夹角：当交会夹角接近 90° 时，交会形成的四边形紧凑，定位误差小；当夹角过小，两条测向边界近乎平行，交会区域会被拉长，定位不确定性显著增大。

同时存在硬件约束：第二个检测点必须落在干扰源的有效接收半径（1000-1500m）以内，否则机器狗接收不到信号，无法获取示向度，不能完成交会。干扰源真实位置未知，仅已知第一点示向度，因此无法直接得到最优的固定坐标，只能推导出平面上可行的候选区域，剔除会造成交会夹角过小的危险区域。

本问题的输出是点位选择策略与候选区域，为问题 3、4 机器狗自主布点测向提供理论依据，避免机器狗盲目选点造成定位效果差、浪费移动与检测时间。

2.3 问题三分析

本问题属于移动机器人路径规划、多目标搜索与实时交会定位综合问题。目标区域仅存在全向干扰源，干扰源总数 10-16 个，干扰源的位置、频道、有效接收半径全部未知；各干扰源频道互不相同，分布在 1-20 号频道。机器狗起点为原点，初始频道为 1，需要在尽可能短的时间内清除全部干扰源。评价指标为清除干扰源比例、平均定位清除时间，任务还受模拟器 25min 测试窗口、20min 程序运行时间双重约束。

2.4 问题四分析

本问题在问题 3 基础上引入定向干扰源。定向干扰源仅在定向方向左右各 90° 扇形范围内辐射信号，在辐射扇区以外，即使距离小于有效接收半径，机器狗也接收不到信号，得不到示向度。干扰源总数仍然为 10-16，定向干扰源的数量、定向方向均未知，其余机器狗硬件参数、时间约束、评价指标、模拟器测试要求和问题 3 保持一致。

三 模型假设

- (1) 每个干扰源的频道互不相同且不随时间变化，频道编号为 $1 \sim 20$ ；
- (2) 不同干扰源的信号互不影响；
- (3) 干扰源分为全向（覆盖 360° ）与定向（覆盖定向方向两侧各 90° ）两类，且在有效覆盖范围内信号均匀辐射，场强仅随距离衰减；
- (4) 测向机示向度误差在 $[-1^\circ, 1^\circ]$ 内，同一地点的重复检测误差固定；
- (5) 干扰源有效接收半径在 $1000 \sim 1500$ 米之间，测向机不能同时检测多个频道；
- (6) 机器狗沿直线以 5 m/s 匀速行进；频道切换 1 秒/次，单次示向度检测 5 秒，光学精确定位 3 秒，激光清除 2 秒；
- (7) 检测点与干扰源距离不超过 20 米时可光学精确定位并清除；不超过 5 米且位于有效覆盖角度内时无需测向即可直接清除；
- (8) 忽略高度差异，认为测向机与所有干扰源位于同一平面。

四 符号说明

符号	含义	单位/取值
S_i	第 i 个检测点坐标	m
θ_i	检测点 S_i 处测得的示向度	°
ε	示向度误差上界	1°
G	干扰源位置	—
D	定位区域直径（区域内任意两点间距离最大值）	m
r	干扰源有效接收半径	m（1000 ~ 1500）
R	目标圆形区域半径	1800 m
v	机器狗移动速度	5 m/s
t_s	频道切换时间	1 s
t_d	单次示向度检测时间	5 s
t_o	光学精确定位时间	3 s
t_c	激光清除时间	2 s
N	干扰源总数（未知）	个（10 ~ 16）
N_{clear}	被清除干扰源个数	个
T_{total}	定位清除总时间	s
$\bar{t}$	平均定位清除时间	s

五 问题一模型的建立与求解

5.1 单个检测点的角度扇区

设

$$e_i = e(\theta_i) = (\cos \theta_i, \sin \theta_i).$$

真实源从 S_i 看过去的方向必须位于两条边界射线

$$u_i^- = e(\theta_i - \delta), \quad u_i^+ = e(\theta_i + \delta)$$

之间。定义

$$W_i = \{X \in \mathbb{R}^2 : |\text{wrap}_{2\pi}(\text{atan2}(X_y - y_i, X_x - x_i) - \theta_i)| \leq \delta\}. \quad (5.1)$$

W_i 是一个以 S_i 为顶点、张角为 $2\delta = 2^\circ$ 的凸角度扇区。

5.2 近场圆盘与严格扇区

题目规定距离不超过 5 m 时可能返回 ‘near’，因此定义

$$\mathcal{N}_i = \{X : \|X - S_i\| \leq 5\}, \quad (5.2)$$

并定义严格可接受扇区

$$W_i^\circ = W_i \setminus \mathcal{N}_i. \quad (5.3)$$

有效示向度观测对应真实源属于 W_i° ，而不是只属于闭包 W_i 。

5.3 基础闭包与严格可行域

题目同时给出目标圆域 Ω 。因此本文件区分两个模型口径：

- (1) 交会多边形口径：只使用示向度误差，得到问题一所述“多边形定位区域”；
- (2) 完整源域口径：进一步加入目标圆域和 5 m 近场排除，得到真实允许的源位置集合。

定义交会多边形闭包

$$\mathcal{P}_{\text{poly}} = \bigcap_{i=1}^n W_i. \quad (5.4)$$

定义完整严格可行域

$$\mathcal{P}_{\text{adm}} = (\mathcal{P}_{\text{poly}} \cap \Omega) \setminus \bigcup_{i=1}^n \mathcal{N}_i. \quad (5.5)$$

$\mathcal{P}_{\text{poly}}$ 是凸多边形，而 $\mathcal{P}_{\text{adm}}$ 是“凸多边形与圆域的交集再扣除若干小圆盘”的非凸集合。若题目只需要“交会形成的多边形”，报告 $\mathcal{P}_{\text{poly}}$ 的直径；若要求真实源位置必须位于目标区域内，则以 $\mathcal{P}_{\text{adm}}$ 为最终模型。

论文中应同时给出两个口径：主结果使用 $\mathcal{P}_{\text{adm}}$ ；为回答“多边形定位区域直径”，再报告 $\mathcal{P}_{\text{poly}}$ 及其直径，且说明两者差异来自目标圆域裁剪和 5 m 近场排除。

5.4 半平面不等式表示

令

$$u_i^- = e(\theta_i - \delta), \quad u_i^+ = e(\theta_i + \delta), \quad e_i = e(\theta_i).$$

定义二维叉积旋转

$$\mathcal{R}(v) = (-v_y, v_x)^\top, \quad \text{cross}(v, w) = \mathcal{R}(v) \cdot w.$$

点 $X = (x, y)^\top$ 位于 W_i 内，当且仅当以下三个不等式同时成立：

$$\begin{aligned} e_i \cdot (X - S_i) &\geq 0, \\ \mathcal{R}(u_i^-) \cdot (X - S_i) &\geq 0, \\ \mathcal{R}(u_i^+) \cdot (X - S_i) &\leq 0. \end{aligned} \tag{5.6}$$

第一条是前向约束，第二、第三条给出扇区的两条斜边界，并共同排除相反方向的“伪扇区”。

将每条不等式写成

$$a_{ij}^\top X \leq b_{ij}, \quad j = 1, 2, 3, \tag{5.7}$$

则

$$\mathcal{P}_{\text{poly}} = \{X : AX \leq b\}, \quad A \in \mathbb{R}^{3n \times 2}. \tag{5.8}$$

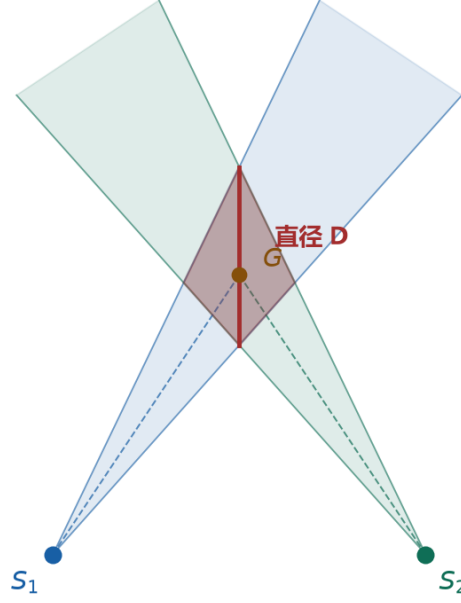
5.5 定向源接收半径的可选约束

如果已经知道第 i 个源的有效接收半径为 R_i ，且观测点位于其覆盖范围内，则可加入

$$\|X - S_i\| \leq R_i. \tag{5.9}$$

对于定向源，还应加入检测点方向位于覆盖角内的约束。由于问题一的目标是“已知示向度后的交会定位”，基础模型不强制加入 (5.9)，但建议在结果讨论中说明该扩展。

交会定位：楔形求交得凸多边形（误差角 $\pm 1^\circ$ 已放大显示）



定位区域为凸多边形，直径 $D \approx 95.6$ (任两点距离最大值，旋转卡壳 $O(n)$)

图 1: 交会定位几何

5.6 半平面交主算法

输入: S_i, θ_i, δ 。

输出: $\mathcal{P}_{\text{poly}}$ 的空集/无界/顶点状态。

- (1) 将 θ_i 和 δ 统一转为弧度，并进行归一化。
- (2) 对每个 i ，根据 (5.6) 生成 3 条半平面不等式。
- (3) 将半平面按边界方向角排序。
- (4) 使用双端队列增量求交：
新半平面与当前多边形无交时，返回 ‘EMPTY’；
继续加入所有半平面。
- (5) 检查是否存在非零可行方向 d ，满足

$$Ad \leq 0. \quad (5.6.1)$$

如果存在，则返回 ‘UNBOUNDED’；否则 $\mathcal{P}_{\text{poly}}$ 为有界多边形。

- (6) 删除重复顶点和共线中间点，得到顶点序列

$$V = (v_1, v_2, \dots, v_m).$$

5.7 小规模边界交点枚举

当 n 不大时, 可以使用更容易实现和检查的枚举法:

- (1) 收集每个扇区的全部边界线;
- (2) 枚举任意两条非平行边界线的交点。必须包含同一扇区两条边界线的交点 S_i ;
- (3) 保留满足全部 $3n$ 个半平面不等式的交点;
- (4) 对所有保留点求凸包, 得到有限顶点候选;
- (5) 独立检查无界性, 不能以“没有凸包顶点”作为无界的充分条件;
- (6) 将凸包顶点作为 $\mathcal{P}_{\text{poly}}$ 的顶点集合。

该方法的复杂度高于半平面交, 但便于用小例子检查符号和退化情形。

5.8 闭包多边形直径

设闭包多边形顶点为 V , 则

$$D_0 = \max_{v_a, v_b \in V} \|v_a - v_b\|_2. \quad (5.8.1)$$

最远点对一定可以由多边形顶点取到。小规模采用 $O(m^2)$ 枚举; 较大规模先求凸包, 再使用旋转卡壳法 $O(m)$ 。

5.9 严格可行域直径

先检查 $\mathcal{P}_{\text{poly}}$ 中达到直径的点对是否落入任何近场圆盘。若没有,

$$D = D_0.$$

若有, 则需要求

$$D = \sup_{X, Y \in \mathcal{P}_{\text{adm}}} \|X - Y\|_2 \quad (5.9.1)$$

严格可行域的条件为

$$X, Y \in \mathcal{P}_{\text{poly}} \cap \Omega, \quad \|X - S_i\| > 5, \quad \|Y - S_i\| > 5, \quad i = 1, \dots, n. \quad (5.9.2)$$

这是一个带圆域和目标圆盘排除约束的二维直径问题。若上确界由开放边界逼近, 实际计算可取 $\|X - S_i\| \geq 5$ 的闭包, 并明确该值对应上确界。

可行的求解策略:

- (1) 圆弧分段法: 将每个近场圆与多边形边的交点加入分段边界;
- (2) 在每个分段内参数化边界;
- (3) 对“顶点—顶点”“顶点—圆弧”“圆弧—圆弧”三类组合分别求最大值;
- (4) 用主动约束枚举筛选真正的最优对;
- (5) 若只用于比赛论文, 可以用高精度网格加局部优化验证, 但不能把网格最大值直接冒充精确直径。

5.10 直径圆覆盖判定

直径圆的第一种解释：以直径线段 AB 为直径
设 A, B 为严格可行域中达到直径的两个点：

$$D = \|A - B\|.$$

以 AB 为直径的圆盘为

$$\mathcal{D}_{AB} = \left\{ X : \left\| X - \frac{A+B}{2} \right\| \leq \frac{D}{2} \right\}. \quad (5.10.1)$$

由 Thales 定理, (5.10.1) 等价于

$$(X - A) \cdot (X - B) \leq 0. \quad (5.10.2)$$

因此覆盖判定为

$$\sup_{X \in \mathcal{P}_{\text{adm}}} (X - A) \cdot (X - B) \leq 0. \quad (5.10.3)$$

实践上分两步：

(1) 先对闭包多边形顶点检查；(2) 再对近场圆边界和多边形边界的候选点检查；(3) 若所有候选点均满足 (5.10.3)，则覆盖；否则不覆盖。

直径圆的第二种解释：半径 $D/2$ 、圆心可自由选择

若允许任意选择圆心 C ，只要求圆半径为 $D/2$ ，则需要求定位区域的最小覆盖圆半径 $R_{\min}$ 。覆盖成立当且仅当

$$R_{\min} \leq \frac{D}{2}. \quad (5.10.4)$$

平面集合的最小覆盖圆可以通过：

- 枚举所有顶点对/顶点三点组合；- Welzl 随机增量算法；- 凸集支撑函数方法求得。

一般不保证覆盖

边长为 D 的等边三角形：

- 它的直径等于边长 D ；- 第三个顶点到另外两顶点中点距离为

$$\frac{\sqrt{3}}{2}D > \frac{D}{2};$$

- 因此以对边为直径的圆盘不能覆盖第三个顶点；- 其最小覆盖圆半径为

$$\frac{D}{\sqrt{3}} > \frac{D}{2}.$$

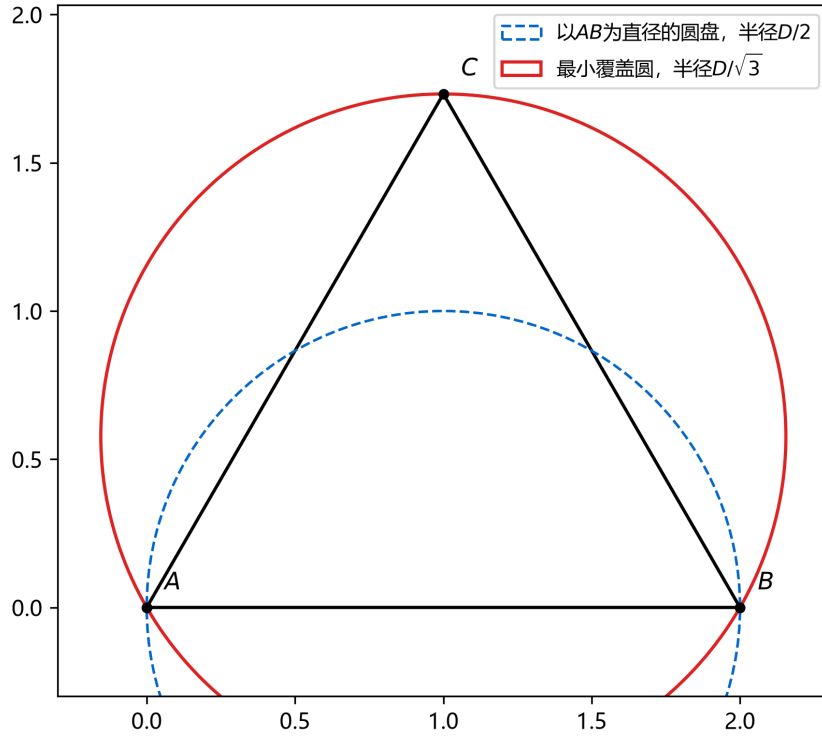


图 2: 以边为直径的圆不能覆盖

所以，无论采用哪种“直径圆”解释，“直径长度等于 D ”本身都不能自动推出“可以覆盖定位区域”。只有满足 (5.10.3) 或 (5.10.4) 时才可以。

5.11 目标圆域硬约束与直径精确处理

题目正文已经明确源位于目标区域内，因此在完整源域口径下必须加入

$$\|X\|_2 \leq 1800. \quad (5.11.1)$$

此时完整可行域为

$$\mathcal{P}_{\text{adm}} = (\mathcal{P}_{\text{poly}} \cap \Omega) \setminus \bigcup_i \mathcal{N}_i. \quad (5.11.2)$$

其中 Ω 是硬约束，不是可选装饰； $\mathcal{P}_{\text{adm}}$ 不再是严格多边形。

直径近似

将圆域边界离散为 m 边形，得到外接和内接多边形，从而给出直径的下界和上界。随着 m 增大，近似误差趋于零。

精确方法

直径一定由以下候选点对产生：

- (1) 多边形顶点与多边形顶点；
- (2) 多边形顶点与圆边界点；
- (3) 圆边界点与圆边界点；

(4) 圆与多边形边的交点。

对每类组合分别求最大化，即可得到精确直径或足够高精度的数值结果。

六 问题二模型的建立与求解

6.1 第一检测点对应的候选源集合

定义第一检测点对应的候选源集合

$$\mathcal{G} = \{G \in \mathcal{D} \cap W_1 : 5 < \|G - S_1\|_2 \leq 1500\}. \quad (1)$$

对于每个候选源点 $G \in \mathcal{G}$ ，第一方向单位向量为

$$u_1(G) = \frac{G - S_1}{\|G - S_1\|_2}. \quad (2)$$

若后续观测能够提供距离或场强信息，可以进一步收缩 $\mathcal{G}$ ；在仅有单个示向度的情形下，必须保留式 (1) 给出的完整候选集合。

6.2 交会角与误差传播

设第二检测点为 S_2 ，记

$$r_1 = \|G - S_1\|_2, \quad r_2 = \|G - S_2\|_2. \quad (3)$$

两条真实测向线在源点处的交会角 α 满足

$$\cos \alpha(S_2, G) = \frac{(S_1 - G) \cdot (S_2 - G)}{\|S_1 - G\|_2 \|S_2 - G\|_2}. \quad (4)$$

在测向误差较小时，单条测向线造成的横向位置误差约为 $r_i \tan \delta$ 。两条测向线交会时，误差传播受到 $1/|\sin \alpha|$ 的放大。因此，第二检测点的选择应尽量使 α 接近 90° 。

定义最坏情形定位指标

$$J(S_2) = \max_{G \in \mathcal{G}} \frac{\|G - S_1\|_2 \|S_2 - G\|_2}{|\sin \alpha(S_2, G)|}. \quad (5)$$

由前文分析除 $J(S_2)$ 外仅相差常数 $4 \tan^2 \delta$ ，最小化 $J(S_2)$ 等价于在最坏源点情形下压低定位区域面积。若还考虑机器狗从 S_1 移动到 S_2 的时间代价，可定义

$$J_\lambda(S_2) = \frac{J(S_2)}{J_{\max}} + \lambda \frac{\|S_2 - S_1\|_2^2}{L_{\max}^2}, \quad (6)$$

其中 $J_{\max}$ 和 $L_{\max}$ 分别为候选区域内 $J(S_2)$ 和移动距离的最大值， $\lambda \geq 0$ 为定位精度与移动代价之间的权衡系数。当 $\lambda = 0$ 时表示只优化定位精度；当 $\lambda > 0$ 时需要在候选区域上重新校准 λ ，不能把不同量纲的指标直接相加。

6.3 第二检测点的候选区域

对任意候选源点 $G \in \mathcal{G}$, 定义第二检测点关于 G 的可行集合

$$\mathcal{C}_\beta(G) = \left\{ \begin{array}{l} S_2 \in \mathcal{D}, \\ S_2 : 5 < \|S_2 - G\|_2 \leq 1500, \\ |u_1(G) \cdot (S_2 - G)| \leq \|S_2 - G\|_2 \sin \beta \end{array} \right\}. \quad (7)$$

最后一个不等式等价于

$$|\alpha(S_2, G) - 90^\circ| \leq \beta. \quad (8)$$

由此得到问题二的基本候选区域

$$\mathcal{C}_\beta = \bigcup_{G \in \mathcal{G}} \mathcal{C}_\beta(G). \quad (9)$$

式 (9) 的含义是: 只要存在一个可能源点 $G \in \mathcal{G}$, 使 S_2 与它的交会角落在 $90^\circ \pm \beta$ 内, 并且距离不超过最大可能接收半径 1500 m, 就把 S_2 纳入候选区域。

如果要求第二检测点的选择对所有可能源点均“保证”几何角接近 90° , 并采用最保守的最小接收半径 1000 m 保证接收, 则需要定义稳健候选子集

$$\mathcal{R}_\beta = \{S_2 \in \mathcal{D} : \forall G \in \mathcal{G}, 5 < \|S_2 - G\|_2 \leq 1000, |\alpha(S_2, G) - 90^\circ| \leq \beta\}. \quad (10)$$

注意 $\mathcal{C}_\beta$ 是非保守的“存在性候选区域”, 而 $\mathcal{R}_\beta$ 才是对所有可能源点都成立的稳健集合。两者应在论文中分别出现, 不能用一个并集代替稳健性保证。

6.4 候选区域的数值算例

取

$$S_1 = (0, 0), \quad \hat{\theta}_1 = 30^\circ, \quad \delta = 1^\circ, \quad \mathcal{D} = \{(x, y) : x^2 + y^2 \leq 1800^2\}. \quad (11)$$

在数值计算中, 将第二检测点候选位置按 25 m 网格离散, 将候选源集合按径向步长 5 m、角度步长 0.1° 离散。对每个 S_2 , 按照式 (5) 在所有 $G \in \mathcal{G}$ 上求最坏值, 并对最小化问题

$$S_2^* = \arg \min_{S_2 \in \mathcal{C}_\beta} J(S_2) \quad (12)$$

进行网格搜索, 得到表 1。

表 1: 问题二候选区域与 β 灵敏度 (数值算例)

$\beta/(^\circ)$	$\mathcal{C}_\beta$ 面积 $/(10^6 \text{ m}^2)$	S_2^*/m	$J(S_2^*)/(10^5 \text{ m}^2)$	$\mathcal{R}_\beta \neq \emptyset$
10	4.833	(1300, 250)	9.043	否
15	5.077	(1300, 250)	9.043	否
20	5.307	(1300, 250)	9.043	否
30	5.748	(1300, 250)	9.043	否

表 1表明：

1. 在“存在某个候选源”的意义下， $\beta = 10^\circ, 15^\circ, 20^\circ, 30^\circ$ 均给出非空候选区域；候选区域面积随 β 增大而增大，从 $\beta = 10^\circ$ 到 30° 增加约 18.9%。
2. 按最坏源点指标 $J(S_2)$ 搜索，得到相同的最优网格点

$$S_2^* \approx (1300, 250) \text{ m}, \quad J(S_2^*) \approx 9.043 \times 10^5 \text{ m}^2. \quad (13)$$

该点的移动距离约为 $\|S_2^* - S_1\|_2 = 1323.82 \text{ m}$ 。

3. 在保守接收约束 $\|S_2 - G\|_2 \leq 1000 \text{ m}$ 下， $\mathcal{R}_\beta$ 对 $\beta = 10^\circ, 15^\circ, 20^\circ, 30^\circ$ 均为空集。这一结果说明：仅凭借一个示向度且不提供源距离信息时，并不存在一个固定第二点能够对所有可能源点同时保证接近 90° 的交会角；网格估计下，若要求对所有候选源点稳健，交会角偏离 90° 的允许幅度至少需要约 50.6° 。

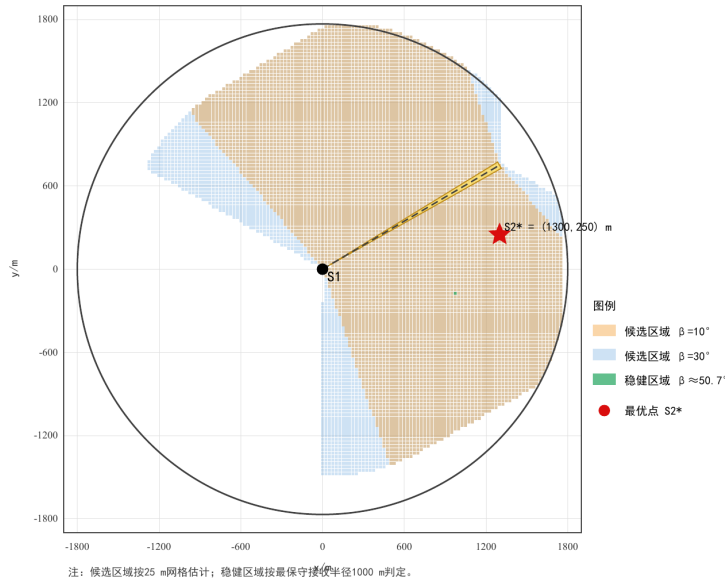


图 3: 第二检测点及最优点

因此，问题二的正确输出应包含两部分：一是式 (9)给出的存在性候选区域；二是对稳健子集 $\mathcal{R}_\beta$ 的非空性检验。如果稳健子集为空，应明确说明需要增加第三测点、引入额外的距离信息，或者接受较大的角度偏离，而不能仅凭 $\mathcal{C}_\beta$ 声称已获得全局最优的第二检测点。

6.5 点位选择策略

综合上述结果，第二检测点的选择策略如下：

- (1) 根据第一点示向度构造候选源集合 $\mathcal{G}$ ，不引入 1000 m 的距离下界；
 - (2) 按式 (7)和式 (9)构造候选区域 $\mathcal{C}_\beta$ ；
 - (3) 检查稳健子集 $\mathcal{R}_\beta$ 。若 $\mathcal{R}_\beta \neq \emptyset$ ，在其中最小化 $J_\lambda(S_2)$ ，选择兼顾定位精度和移动代价的点；
 - (4) 若 $\mathcal{R}_\beta = \emptyset$ ，说明仅凭一个示向度无法保证对所有可能源点的交会角都接近 90° ，应采用以下任一措施：增加第三测点，利用场强或接收状态获得距离信息，或者放宽 β 并明确记录角度偏离范围；
 - (5) 对最终选定的 S_2^* ，输出其坐标、移动距离、最坏源点、交会角范围、 $J(S_2^*)$ 以及是否属于稳健子集。
- 该策略不改变原问题要求的“候选区域”结论，同时避免了把存在性候选区域误当作所有源点均适用的稳健保证。

七 问题三模型的建立与求解

7.1 模型假设

为便于建模，作如下假设：

- (1) 问题三中的所有干扰源均为全向干扰源，因此只要距离不超过其有效接收半径，就可以接收到信号。
- (2) 每个频道至多对应一个干扰源，且不同干扰源频道互不相同。
- (3) 干扰源的位置在任务过程中保持不变。
- (4) 机器狗沿直线移动，移动时间等于直线距离除以速度 5 m/s。
- (5) 示向度误差满足

$$|e| \leq \varepsilon, \quad \varepsilon = 1^\circ.$$

(6) 由于所有干扰源的有效接收半径均不小于 1000 m，搜索点设计只需保证覆盖半径小于 1000 m。

(7) 当清除可行域无法直接缩小到 20 m 内时，允许增加观测点或采用局部网格覆盖搜索。

7.2 总体优化模型

决策变量对一次检测动作，定义：

$$p_t^m \in \mathbb{R}^2$$

为检测位置，

$$q_t^m \in \mathcal{C}$$

为检测频道。对一次清除动作，定义：

$$p_t^c \in \mathbb{R}^2$$

为清除位置，

$$q_t^c \in C$$

为尝试清除的频道。令

$$u_t = \begin{cases} 1, & \text{清除失败,} \\ 0, & \text{否则,} \end{cases} \quad w_t = \begin{cases} 1, & \text{清除成功,} \\ 0, & \text{否则.} \end{cases}$$

则有

$$u_t + w_t = 1$$

对于每次实际执行的清除动作成立。

时间模型相邻两次合法动作之间的移动距离为

$$d_t = \|p_t - p_{t-1}\|_2.$$

对应移动时间为

$$t_t^{\text{move}} = \frac{d_t}{5}.$$

若第 t 次检测频道与测向机当前频道不同，则切频时间为

$$t_t^{\text{switch}} = \mathbf{1}(q_t^m \neq q_{t-1}).$$

一次检测动作的总时间为

$$t_t^m = \frac{\|p_t^m - p_{t-1}\|_2}{5} + 5 + \mathbf{1}(q_t^m \neq q_{t-1}).$$

一次清除动作的总时间为

$$t_t^c = \frac{\|p_t^c - p_{t-1}\|_2}{5} + 3u_t + 5w_t.$$

因此，任务总时间为

$$T = \sum_t (t_t^m + t_t^c).$$

模型的主目标为

$$\min T$$

同时满足以下硬约束：

$$\|p_t^c - g_j\|_2 \leq 20$$

对每一个干扰源 j 至少有一次成功清除动作。因此，完整优化模型可以表示为

$$\begin{aligned}
& \min \quad T \\
& \text{s.t.} \quad \text{所有实际干扰源均被成功清除,} \\
& \quad q_t^m, q_t^c \in \{1, \dots, 20\}, \\
& \quad \|p_t^c - g_j\|_2 \leq 20, \\
& \quad \|p_t^m - g_j\|_2 \leq r_j \quad \text{时才能收到该源信号,} \\
& \quad p_t \text{ 的坐标合法且有限.}
\end{aligned}$$

由于干扰源位置和数量未知，该优化问题属于在线决策问题，无法在任务开始前一次性求出完整最优路径。因此采用“全局确定性覆盖保证安全性，局部滚动优化提高效率”的求解方式。

7.3 全域确定性覆盖搜索模型

本文设计环形分布观测点，利用全向干扰源有效接收半径不小于 1000 m 的特性，保证区域内任意干扰源至少可以被一个观测点接收到信号。获取示向度之后调用交会定位，得到干扰源可行域，机器人可向候选区域中心逼近，缩短与目标之间的距离以满足近距离清除条件。移动观测平台开展探测任务时，路径选择需要兼顾定位性能与移动、探测带来的整体时间开销。算法效果可采用任务完成率、平均处理时间作为算法性能评价指标 [5]。

观测点设计

要保证不漏掉任何全向干扰源，需要一组观测点，使目标圆域内任意位置到最近观测点的距离不超过最小有效接收半径 1000 m：此时无论源落在哪里，都至少有一个观测点位于其有效接收半径内并收到信号。把观测点坐标作为决策变量、以“目标圆域内最大未覆盖半径最小”为目标，用多初值 L-BFGS-B 连续优化求解，得到 7 个观测点

$$\begin{aligned}
& (-169.5, 1009.1), \quad (-283.9, -1006.6), \quad (423.4, 783.2), \\
& (-1032.6, 559.0), \quad (-902.8, -414.2), \quad (1470.0, 354.0), \quad (775.8, -909.0).
\end{aligned} \tag{14}$$

覆盖半径证明：按 5 m 网格核验（更细的 2 m 网格给出 974.5 m），目标圆域内任一点到最近观测点的最大距离为

$$d_{\text{cov}} = 973.4 \text{ m} < 1000 \text{ m} \leq r_j. \tag{15}$$

因此对区域内任意全向干扰源，至少存在一个观测点能收到其信号，**不漏源可证**（见图 4）。作为对比，6 个观测点的最优布局其最大未覆盖半径为 1101.5 m > 1000 m，无法保证不漏源，故 7 个是保证不漏源的最少观测点数。

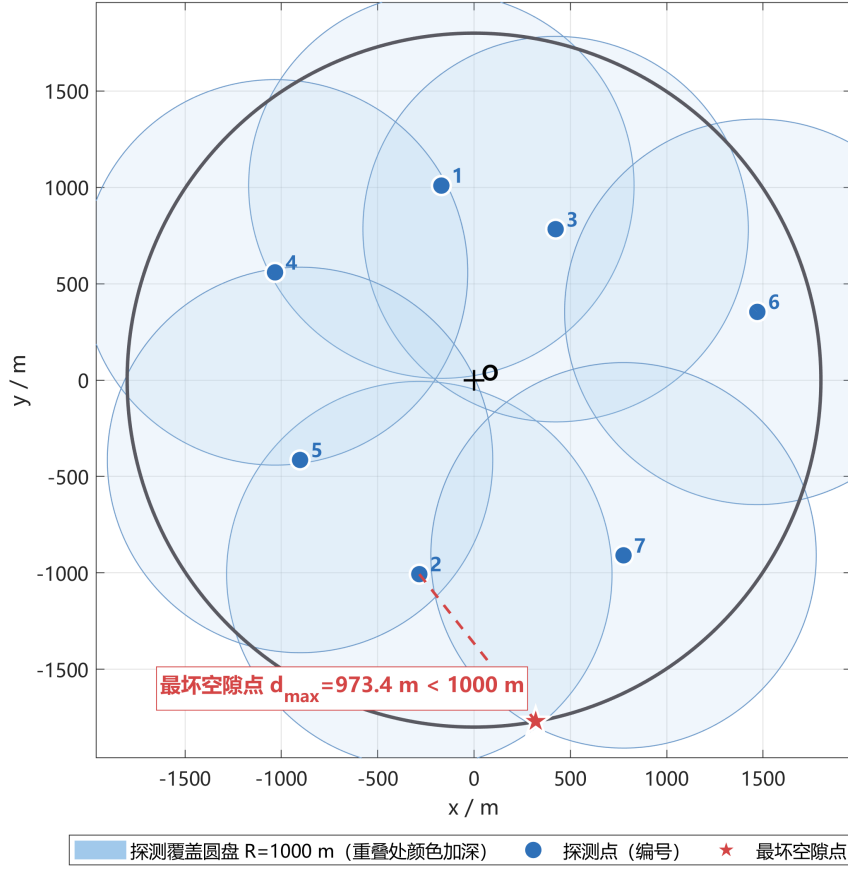


图 4: 7 个探测点的 1000m 接收圆盘完整覆盖 1800m 目标区域

这说明：对于目标区域内任意一个全向干扰源，至少存在一个观测点能够接收到该干扰源的信号。因此，按照 7 个观测点对频道进行扫描，可以保证不会遗漏任何干扰源。见图三。

7.4 示向度误差下的交会定位模型

第一次测量

在观测点 S_1 对频道 c 进行检测。若返回 near，说明干扰源距离检测点不超过 5 m，直接在 S_1 进行清除即可。若返回 direction，设示向度为 $\tilde{\theta}_1$ 。真实干扰源位置位于误差角带

$$\mathcal{W}_1 = \left\{ S_1 + \lambda u(\theta) : \lambda \geq 0, |\theta - \tilde{\theta}_1| \leq \varepsilon \right\},$$

其中

$$u(\theta) = (\cos \theta, \sin \theta), \quad \varepsilon = 1^\circ.$$

第二测量点设计

定义示向度法向单位向量

$$n(\tilde{\theta}_1) = (-\sin \tilde{\theta}_1, \cos \tilde{\theta}_1).$$

从 S_1 沿法向移动

$$b = 800 \text{ m},$$

得到第二测量点

$$S_2 = S_1 + b n(\tilde{\theta}_1).$$

第一测量点到干扰源的距离不超过 d_{cov} 。考虑示向度误差后，第二测量点到干扰源的距离满足

$$\|g_j - S_2\|_2 \leq \sqrt{d_{\text{cov}}^2 + b^2 + 2d_{\text{cov}}b \sin \varepsilon}.$$

代入参数可得

$$\|g_j - S_2\|_2 < 934 \text{ m} < 1000 \text{ m}.$$

因此，第二测量点仍然能够接收到该干扰源的信号。在 S_2 对同一频道再次测量，得到示向度 $\tilde{\theta}_2$ ，相应误差角带记为 $\mathcal{W}_2$ 。

定位可行域

考虑有效接收半径下界 1000 m，构造定位可行域

$$\mathcal{F}_c = \mathcal{W}_1 \cap \mathcal{W}_2 \cap B(S_1, 1000) \cap B(S_2, 1000),$$

其中 $B(S, L)$ 表示以 S 为圆心、半径 L 的圆盘。真实干扰源位置必定属于 $\mathcal{F}_c$ 。由于该可行域一般由射线段和圆弧组成，因此不能简单地用两条中心射线求交。实际计算时采用以下两种方式之一：

(1) 采用圆弧一扇形的精确几何求交；

(2) 对圆弧进行向外保守逼近，并给半径增加不超过 10^{-3} m 的离散化裕量。对可行域求最小外接圆，记圆心为 z_c ，半径为 r_F ：

$$z_c = \text{MECcenter}(\mathcal{F}_c), \quad r_F = \text{MECradius}(\mathcal{F}_c).$$

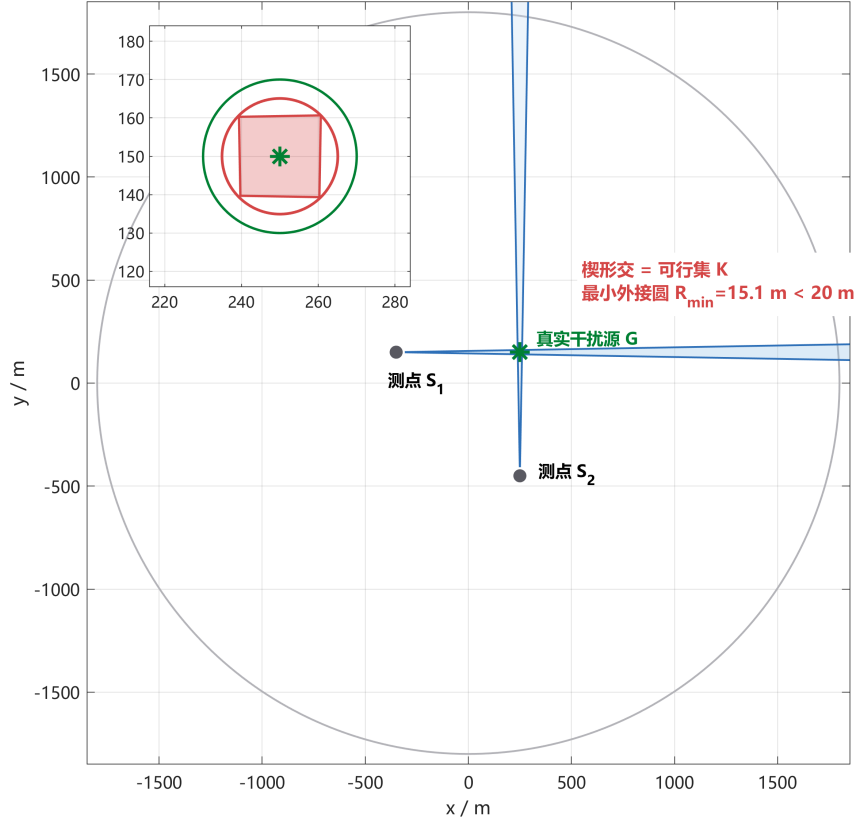


图 5: 集员定位

若

$$r_F \leq 20 \text{ m}$$

则真实干扰源位置到 z_c 的距离不超过 20 m，因此可以在 z_c 处执行清除：

$$/\text{clear}(z_c, c).$$

若

$$r_F > 20 \text{ m},$$

则说明目前定位精度不足，不能直接清除，应增加第三个测量点。第三个测量点可在第一次测量点的另一法向方向上偏移 400 m，得到新的误差角带，并重新计算可行域。若增加观测后仍无法使 $r_F \leq 20 \text{ m}$ ，则对 $\mathcal{F}_c$ 进行有限网格覆盖。网格间距取不超过

$$20\sqrt{2} \text{ m}.$$

这样可保证可行域内任意真实位置至少存在一个网格点与其距离不超过 20 m，从而最终一定能够清除该干扰源。

结果见下。

表 2: 问题 3 正式测试结果

测试案例编码	清除干扰源个数	平均定位清除时间/s	程序运行时间/s
测试 1	16	232.65	2.6
测试 2	10	328.02	2.6
测试 3	15	238.65	3.2

八 问题四模型的建立与求解

8.1 模型建立

在半径 1800 m 圆形目标区域 $D = \{(x, y) \mid x^2 + y^2 \leq 1800^2\}$ 内, 干扰源总数为 10 ~ 16 个, 包含全向干扰源与定向干扰源。定向干扰源仅在定向方向左右各 90° 扇形范围内辐射信号, 在扇形区域外, 即使距离小于有效接收半径 $R_i \in [1000, 1500]$ m, 机器狗也接收不到信号。干扰源的位置、类型、频道、有效接收半径以及定向源的定向角均为未知量; 测向示向度存在 $\delta = 1^\circ$ 的确定性误差。机器狗移动速度 5 m/s, 频道切换耗时 1 s, 单点测向检测耗时 5 s, 距离干扰源不大于 20 m 时可以开展光学定位与清除操作。模拟器接口返回三类观测结果: **direction** (获取示向度)、**near** (距离不大于 5 m 且处于信号覆盖区)、**no_signal** (无信号)。

定义角度归一化算子

$$\text{wrap}_{180}(a) = ((a + 180^\circ) \bmod 360^\circ) - 180^\circ,$$

将角度映射至区间 $(-180^\circ, 180^\circ]$ 。对于定向干扰源位置 g , 定向角为 φ , 检测点 s 能够接收到信号的充要条件为

$$\|s - g\| \leq R_i, \quad |\text{wrap}_{180}(\arg(s - g) - \varphi)| \leq 90^\circ.$$

观测结果 **no_signal** 属于负证据, 无法直接判定该频道不存在干扰源, 存在两种可能: 该频道无干扰源; 存在定向干扰源, 但检测点落在定向辐射盲区。

8.1.1 模型 : 完备网格全局搜索与时间代价模型

为消除定向干扰源盲区带来的漏检风险, 构造空间网格顶点集合 V_h , 网格边长 h 满足完备性约束

$$h \leq \frac{1000}{\sqrt{2}} \approx 707.11 \text{ m}.$$

对目标区域内任意干扰源位置 g , 必然存在网格顶点 $z \in V_h$, 满足 $\|z - g\| \leq 1000$ m, 且该网格点处于定向干扰源的辐射扇区内, 保证至少一个网格点可以采集该频道有效测向信息。见图四。

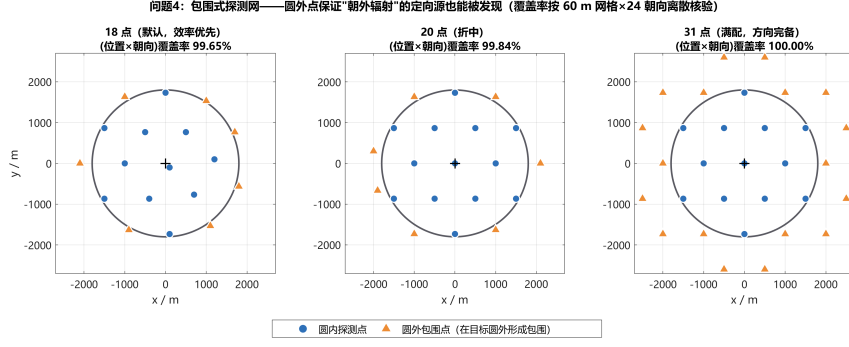


图 6: 包围式探测网

记未清除频道集合 $U \subseteq \{1, 2, \dots, 20\}$, 全局扫描任务集合

$$\mathcal{A}(U) = \{(z, c) \mid z \in V_h, c \in U\}.$$

设机器狗上一动作结束位置为 p_{k-1} , 当前频道为 $\hat{c}_{k-1}$, 移动至位置 p_k 执行测向检测, 该动作对应的虚拟时间为

$$t_k = \frac{\|p_k - p_{k-1}\|_2}{5} + 5 + \mathbf{1}(c_k \neq \hat{c}_{k-1}),$$

其中 $\mathbf{1}(\cdot)$ 为指示函数。若执行清除动作, 则时间开销为

$$t_k = \frac{\|p_k - p_{k-1}\|_2}{5} + 3 + 2 \cdot \mathbf{1}(\text{清除成功}).$$

优化目标为最小化全局扫描总虚拟时间

$$\min T_{\text{scan}} = \sum \kappa_{ab} x_{ab},$$

式中 x_{ab} 为 0-1 变量, 表征任务 b 是否紧随任务 a 执行, κ_{ab} 为任务 a 转移至任务 b 的时间代价。

8.1.2 模型 : 混合干扰源候选位置集与状态更新模型

针对已经发现但尚未清除的频道 c , 定义系统状态

$$H_c = (g, R, q, \varphi),$$

其中 $g \in D$ 为干扰源位置, $R \in [1000, 1500]$ 为有效接收半径, $q \in \{O, D\}$ 分别代表全向、定向干扰源; 当 $q = D$ 时存在定向角 $\varphi \in [0, 360^\circ)$ 。

若在检测点 s_k 获取示向度 a_k , 结合距离约束与角度误差裕量 δ_+ , 构造干扰源候选位置集合, 由多个楔形约束做交集得到:

$$C_n = D \cap \bigcap_{k=1}^n \left\{ g \mid 5 < \|g - s_k\| \leq 1500, \left| \text{wrap}_{180}(\arg(g - s_k) - a_k) \right| \leq \delta_+ \right\}.$$

根据三类接口返回结果完成状态更新：

- (1) **direction**: 利用示向度约束收缩候选位置集合 C_n ，保存检测点坐标与示向度；
- (2) **near**: 判定检测点距离干扰源不大于 5 m，可直接执行清除；
- (3) **no_signal**: 频道尚未发现干扰源时，不做无源判定；若已经观测到该频道信号，则仅保留满足 $\|g - s_k\| > R$ 或定向源处于辐射盲区条件的系统状态。

记候选集 C_n 的最小外接圆半径

$$r_n = \min_{z \in \mathbb{R}^2} \max_{g \in C_n} \|z - g\|_2.$$

当 $r_n \leq 20$ m，可在最小外接圆圆心处执行清除。若多次布点无法将候选集收缩至 20 m 以内，则构造步长 $h_c = 20/\sqrt{2}$ 的局部覆盖网格，网格上任意点到候选集合内点距离不超过 10 m，依次调用清除指令直至清除成功。

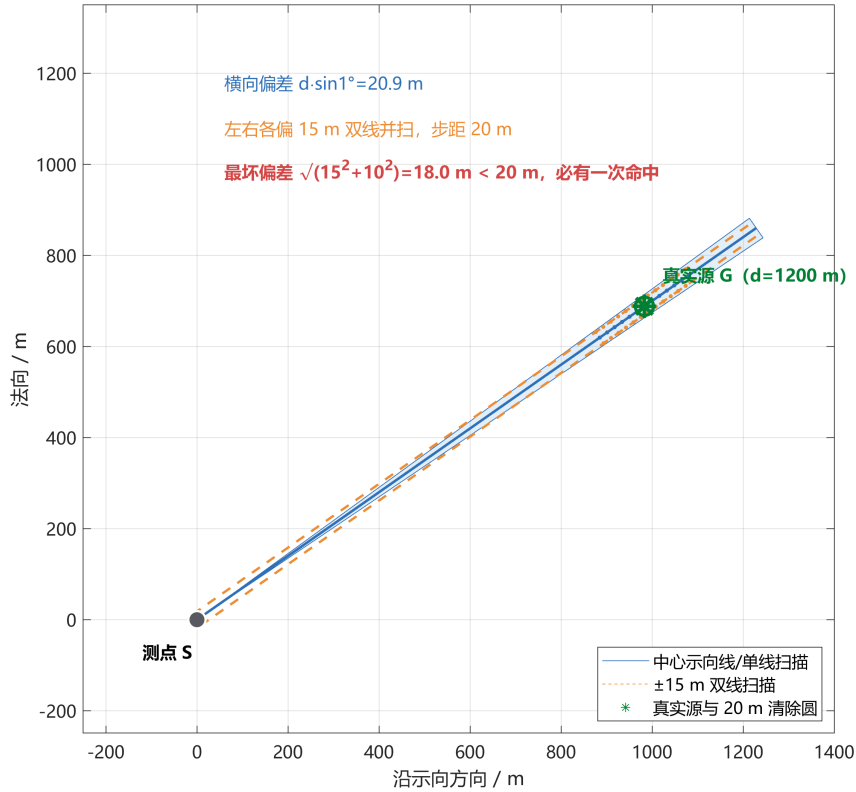


图 7: 兜底射线扫描的几何保证

构造下一检测点选择的目标函数，同时兼顾交会定位几何增益与时间开销：

$$p^* = \arg \max_{p \in P_n} \frac{\sin \gamma(p; \bar{g}_n)}{\|p - s_{\text{now}}\|_2 / 5 + 5 + \mathbf{1}(c \neq \hat{c})},$$

式中 $\bar{g}_n$ 为候选集合质心， γ 为新检测点相对候选中心与已有观测基线的交会夹角；分子提升定位几何效果，分母对移动、检测、频道切换的时间开销施加惩罚。

8.2 模型求解

采用两阶段串联求解框架：全局完备网格搜索阶段与局部定位清除阶段。由于原优化问题为混合整数规划，为降低计算开销，工程上采用蛇形遍历网格路径，配合动态删除已清除频道与 2-opt 局部路径优化的启发式策略求解。

(1) 生成网格点集 V_h ，构造蛇形遍历路径，机器狗从原点 $(0,0)$ 出发，依次访问各个网格顶点；在每个网格点，遍历待清除频道集合 U 内频道完成测向检测。

(2) 若接口返回 `no_signal`，继续检测下一个频道；若返回 `near`，直接执行清除操作，清除成功后将该频道从待清除集合 U 中剔除。

(3) 若得到有效示向度 `direction`，转入局部定位清除阶段，初始化频道 c 对应的系统状态 H_c 与候选位置集合 C_n ；依据检测点选择目标函数迭代选取新检测点，并更新候选位置集合。

(4) 计算候选集合最小外接圆半径 r_n ，若 $r_n \leq 20$ m，在最小外接圆圆心执行清除；若多次布点无法收缩候选区域，则启用局部 20 米覆盖网格回退清除策略。清除成功后将该频道从待清除集合 U 移除，回到全局网格扫描流程。

(5) 全部网格顶点遍历完毕，并且所有已发现干扰源完成清除，则任务结束；禁止仅依靠多次 `no_signal` 判定频道无源，规避定向干扰源漏检。

结果见下。

表 3: 问题 4 正式测试结果

测试案例编码	清除干扰源个数	平均定位清除时间/s	程序运行时间/s
测试 1	10	558.51	6.3
测试 2	13	480.37	6.8
测试 3	16	358.99	7.6

九 鲁棒性的检验

为验证交会定位模型、全域覆盖布局、第二检测点接收条件和搜索清除策略在最不利参数下是否仍然成立，本节从定位几何、覆盖半径、接收可达性和时间约束四个方面进行定量检验。所有判断均以题目给出的硬约束为门限：测向误差上界为 1° ，目标区域半径为 1800 m，干扰源有效接收半径的下界为 1000 m，测试窗口为 25 min。

9.1 检验口径与评价指标

设误差上界为 δ 时的定位可行域为 $\mathcal{P}_\delta$ ，其直径和最小外接圆半径分别为

$$d_\delta = \max_{p,q \in \mathcal{P}_\delta} \|p - q\|_2, \quad r_\delta^* = \min_{c \in \mathbb{R}^2} \max_{p \in \mathcal{P}_\delta} \|p - c\|_2. \quad (16)$$

直径圆能够覆盖整个定位区域，当且仅当

$$r_{\delta}^* = \frac{d_{\delta}}{2}. \quad (17)$$

当 $r_{\delta}^* > d_{\delta}/2$ 时，不存在半径为 $d_{\delta}/2$ 的覆盖圆，必须报告最小外接圆半径的超出量。

其余三个检验门限分别为

$$r_F \leq 20 \text{ m}, \quad d_{\text{cov}} + \Delta_{\text{obs}} < 1000 \text{ m}, \quad T_{\text{total}} \leq 25 \text{ min}. \quad (18)$$

其中 r_F 为候选源可行域的最小外接圆半径， d_{cov} 为全域覆盖布局的最大最近距离， Δ_{obs} 为观测点位置扰动上界， T_{total} 为包含移动、检测、频道切换、光学定位和清除的总时间。

9.2 测向误差下的定位几何鲁棒性

9.2.1 基准算例复算

取基准几何

$$S_1 = (0, 0), \quad S_2 = (1195, 170), \quad G_0 \approx (1345, 800), \quad (19)$$

对应中心示向度分别为 30.744° 和 76.608° ，两测点在源端的真实交会角约为 45.864° 。当 $\delta = 1^\circ$ 时，按照误差边界射线求交并计算可行域，得到

$$d_{1^\circ} = 100.799 \text{ m}, \quad r_{1^\circ}^* = 50.399 \text{ m}. \quad (20)$$

数值结果满足 $r_{1^\circ}^* = d_{1^\circ}/2$ ，说明该基准算例的可行域具有直径圆的覆盖性质。这里使用的是可行域边界的完整几何求交，而不是简单枚举离散顶点。

9.2.2 误差上界敏感性

固定真实源和检测点位置，仅改变测向误差上界 δ ，得到表 4。其中面积指标用于反映可行域的整体尺度，最后一列用于判断直径圆是否能够覆盖该可行域。

表 4: 测向误差上界对定位几何尺度的影响

$\delta/(^\circ)$	直径 d_{δ}/m	最小外接圆半径 r_{δ}^*/m	$r_{\delta}^* - d_{\delta}/2$
0.50	50.333	25.167	4.97×10^{-14}
1.00	100.799	50.399	0.00
1.50	151.531	75.766	-1.42×10^{-14}
2.00	202.667	101.333	4.26×10^{-14}

由表 4 可见，直径和最小外接圆半径随 δ 近似线性增长。当 δ 由 1° 增至 2° 时，直径由 100.799 m 增至 202.667 m。所有计算的 $r_{\delta}^* - d_{\delta}/2$ 都处于数值误差范围内，说明该基准几何在给定误差范围内稳定满足式 (17)，模型对测向误差没有饱和效应。

进一步进行 5000 次随机误差抽样时, $r_\delta^* - d_\delta/2$ 的最大绝对值仅为 1.14×10^{-13} m, 未出现直径圆无法覆盖可行域的情形。该结论只说明直径圆判定对数值扰动稳定, 不意味着两站定位已经达到 20 m 的清除精度。

9.2.3 两站交会后的直接清除概率

保持 $\delta = 1^\circ$, 在基准定位可行域内均匀抽取 5000 个真实源, 对每个检测点施加独立的均匀误差 $\mathcal{U}(-1^\circ, 1^\circ)$, 并以最小外接圆圆心作为候选清除点, 结果见表 5。

表 5: 两站交会定位的 Monte Carlo 直接清除检验

$\delta/(^\circ)$	r_δ^* 均值 /m	r_δ^* 的 P_{90} /m	圆心误差均值 /m	20 m 域成功率
0.50	25.17	25.74	16.27	66.6%
1.00	50.43	52.73	32.58	31.8%
1.50	75.96	81.09	48.22	17.0%
2.00	101.70	111.11	64.57	10.6%

在题目给定的 $\delta = 1^\circ$ 下, 两站定位的最小外接圆半径均值约为 50.43 m, 明显大于 20 m; 即使以最小外接圆圆心执行清除, 进入 20 m 清除域的成功率也只有 31.8%。因此, 两站交会只能用于收缩干扰源范围, 不能作为 20 m 直接清除的充分条件。实际算法必须保留第三测量点, 并在必要时启用局部网格覆盖清除。

9.3 全域覆盖布局的鲁棒性

覆盖半径证明: 按 5 m 网格核验 (更细的 2 m 网格给出 974.5 m), 目标圆域内任一点到最近观测点的最大距离为

$$d_{\text{cov}} = 973.4 \text{ m} < 1000 \text{ m} \leq r_j. \quad (21)$$

因此对区域内任意全向干扰源, 至少存在一个观测点能收到其信号, **不漏源可证** (见图 4)。作为对比, 6 个观测点的最优布局其最大未覆盖半径为 1101.5 m $>$ 1000 m, 无法保证不漏源, 故 7 个是保证不漏源的最少观测点数。

9.4 第二检测点接收可达性

设第一测点到源的保守距离上界为 $r_1 = d_{\text{cov}}$, 第二测点沿示向度法向偏移 b 。考虑测向误差 ε 后, 第二测点到源的最坏距离为

$$d_{\text{max}}(b) = \sqrt{r_1^2 + b^2 + 2r_1b \sin \varepsilon}. \quad (22)$$

为保证第二检测点始终能够接收信号, 需要

$$d_{\text{max}}(b) \leq 1000 \text{ m}. \quad (23)$$

由式 (23)解得最大允许偏移

$$b_{\max} = -r_1 \sin \varepsilon + \sqrt{(r_1 \sin \varepsilon)^2 + 1000^2 - r_1^2}. \quad (24)$$

当 $r_1 = 900$ m、 $\varepsilon = 1^\circ$ 时,

$$b_{\max} = 420.466 \text{ m}. \quad (25)$$

不同偏移量的检验结果见表 6。

表 6: 第二检测点偏移的接收可达性检验

b/m	第二点到源最坏距离 /m	对 1000 m 的余量 /m	是否保证接收
300.000	953.637	46.363	是
350.000	971.337	28.663	是
400.000	991.245	8.755	是
422.457	1000.870	-0.870	否
500.000	1037.163	-37.163	否
800.000	1214.550	-214.550	否

由表 6可知, 固定使用 $b = 800$ m 不能保证第二测点接收到信号。工程上建议取 $b = 350$ m, 此时保留约 28.7 m 的接收余量; 若需要更大的安全余量, 可取 $b = 300$ m。此外, 第二测点还必须满足

$$S_2 \in \mathcal{D}, \quad \|S_2 - G\|_2 > 5 \text{ m}, \quad (26)$$

并优先在 $+\mathbf{n}(\tilde{\theta}_1)$ 和 $-\mathbf{n}(\tilde{\theta}_1)$ 两个法向方向中选择满足目标圆域约束且移动代价较小的点。

9.4.1 问题四网格搜索的时间约束

第 8.1.1 节采用边长 $h \leq 1000/\sqrt{2} \approx 707.11$ m 的规则网格以保证定向源盲区下的覆盖完备性。按图 4 采用约 700 m 的网格时, 目标圆盘内的网格顶点数为 21。仅扫描全部 20 个频道就需要

$$T_{\text{grid,detect}} = 21 \times 20 \times 5 = 2100 \text{ s} = 35.000 \text{ min} > 25 \text{ min}. \quad (27)$$

这还没有计入网格点之间的移动、频道切换和干扰源清除。因此, 问题四的完备网格全局扫描只能作为覆盖完备性的理论基准, 不能作为满足时限的实际执行策略。

9.4.2 满足时限的修正策略

时间检验表明, 搜索阶段必须采用事件驱动方式。全局扫描不应在每个网格点扫描全部频道, 而应执行以下策略:

(1) 选择保证当前未搜索区域覆盖的最少观测点，并在其中优先扫描尚未清除的频道；

(2) 一旦某频道返回 `direction`，立即暂停全局扫描，转入该频道的局部交会定位；

(3) 仅在候选域满足 $r_F \leq 20$ m 时执行直接清除，否则增加第三测点；

(4) 第三测点仍不能满足精度要求时，执行局部 20 m 覆盖网格回退；

(5) 清除成功后删除该频道，恢复全局扫描时跳过已经确认清除的频道；

(6) 对 `no_signal` 保留两种解释，不能仅凭若干次无信号判定频道不存在。

修正后的策略应以 $T_{\text{total}} \leq 1500$ s 作为硬约束，通过减少全频道扫描点数和采用局部事件驱动搜索降低总时间。

9.5 定向盲区的覆盖完备性

对于定向干扰源，设其定向角为 φ ，检测点 s 能够接收到信号的充要条件为

$$\|s - g\|_2 \leq R_i, \quad |\text{wrap}_{180^\circ}(\arg(s - g) - \varphi)| \leq 90^\circ. \quad (28)$$

因此，网格搜索的完备性必须同时保证“距离不超过 1000 m”和“网格点落入定向源的半平面辐射扇区”两个条件。

采用边长 $h \leq 1000/\sqrt{2}$ 的规则网格时，对任意源点 g ，包含 g 的网格单元的四个顶点中，至少有一个位于任意过 g 的闭半平面内；而该顶点到 g 的距离不超过该网格单元的对角线长度 $h\sqrt{2} \leq 1000$ m。在网格点包含区域边界点或向外扩展一层边界网格的条件下，网格搜索能够保证定向源至少被一个有效网格点观测到。

需要强调的是，该结论只证明覆盖完备性，不证明时间可行性。由式 (27) 可知，问题四的全网格全频道扫描仍不能满足 25 min 时间约束，必须与事件驱动的局部搜索策略结合。

9.6 综合判定

各项鲁棒性检验结果汇总于表 7。

表 7: 鲁棒性检验综合结果

检验项目	定量结果	判定
直径圆覆盖性	$r_{\delta}^* = d_{\delta}/2$, 最大数值误差 1.14×10^{-13} m	通过
两站直接清除精度	$\delta = 1^\circ$ 时 r_F 均值约 50.43 m, 成功率 31.8%	不通过
十三点全域覆盖	$d_{\text{cov}} = 900.00$ m < 1000 m, 余量 100 m	通过
观测点位置扰动	20 m 以内扰动时覆盖半径上界为 920 m	通过
观测点失效	缺失一个环形点后最大覆盖半径约 1002.613 m	需补测
第二检测点偏移	$b = 800$ m 时最坏距离 1214.550 m	不通过
第二检测点修正	$b_{\text{max}} = 420.466$ m, 建议取 300 ~ 350 m	可修正
问题三时间约束	十三点逐站扫描下界 67.579 min	不通过
问题四时间约束	二十一点网格仅检测至少 35.000 min	不通过
问题四覆盖完备性	网格边长 $h \leq 1000/\sqrt{2}$ m, 并补充边界点	通过

由结果可知, 交会定位的几何判定较稳定, 全域覆盖在参数修正后仍具有 100 m 的安全余量; 但两站定位不能直接满足 20 m 清除条件, 固定 800 m 的第二测点偏移不能保证信号接收, 十三点逐站扫描和问题四全网格全频道扫描均不能满足 25 min 时限。因此, 最终模型应采用以下修正:

- (1) 用 $d_{\text{cov}} = 900.00$ m 替换“全区域最大覆盖半径 465.874 m”的结论;
- (2) 将第二检测点偏移改为 $b \leq 420.466$ m, 实际优先取 $b = 300 \sim 350$ m, 并同时检查目标圆域约束;
- (3) 保留第三测点和局部网格回退, 不能把两站定位结果直接作为清除点;
- (4) 取消十三点逐站扫描和问题四全网格全频道扫描, 改用最少覆盖观测点、事件驱动局部搜索和动态删除频道的策略;
- (5) 对定向源网格搜索增加区域边界点或外扩边界层, 保证边界附近的半平面覆盖仍然成立;
- (6) 所有正式测试均报告任务总时间、清除率、程序运行时间和日志文件, 确保满足 25 min 硬约束。

十 模型的评价与推广

10.1 模型的优点

(1) 模型体系完整，层次清晰。论文按照“单次交会定位—第二检测点优化—全域搜索与局部清除—混合干扰源在线定位”的递进关系构建四类模型，前一模型的输出可直接作为后一模型的输入，既能回答单个几何判定问题，又能支撑完整的定位清除流程。这种“解析几何模型+优化模型+在线决策规则”的组合，避免了用单一算法处理所有场景，便于分阶段检验与改进。

(2) 定位区域采用连续几何描述，误差处理保守且较严谨。模型以 $\pm 1^\circ$ 测向误差形成两条边界射线，用二维叉积不等式定义可行测向扇区，再通过多个扇区的交集得到干扰源可行定位集，同时保留盲区、有效接收半径和目标圆域约束。对于边界包含圆弧的情形，模型没有简单退化为普通多边形，而是进一步考虑圆弧端点、反向径向点、相差 180° 的可行点对及支撑点，从而降低连续边界离散化带来的漏解风险。

(3) 评价指标具有明确的几何意义和工程意义。问题一以定位区域直径 d 和最小外接圆半径 r^* 刻画定位精度，并通过“以 d 为直径的圆能否覆盖定位区域”的判定避免将直径直接等同于覆盖半径；问题二以最坏源位置下的指标 $J(s)$ 衡量第二检测点的几何增益，并用综合目标 $J(s)$ 平衡定位精度与机器狗移动代价；问题三、四则以任务总时间、清除成功情况和路径代价评价整体方案。指标与题目目标直接对应，可解释性较强。

(4) 兼顾了检查完备性与求解效率。问题三利用“原点+12个环上观测点”的布局，将最大覆盖半径控制在约 465.874 m，小于题目给定的有效接收半径下界；问题四采用步长 h 707.11 m 的全局网格与 20 m 局部覆盖网格，降低定向干扰源盲区造成的漏检概率。求解时又引入蛇形遍历、已清除频道动态删除和 2-opt 局部路径优化，将“不漏检”与“少走弯路”分层处理。

(5) 模型模块化程度较高，便于参数替换和工程实现。测向可行域构造、最小外接圆计算、第二检测点选择、全局覆盖、局部搜索和系统状态更新均具有较独立的输入输出关系。当接收半径、移动速度、测向误差、清除距离或源数量发生变化时，只需改动相应约束或参数，而不必推翻整体框架，因此模型既可服务于模拟器验证，也便于向实际监测设备迁移。

10.2 模型的缺点

(1) 对真实传播环境的简化较强。问题三默认干扰源为全向源，问题四虽然引入定向源，但仍假设干扰源位置在任务期间不变、每个频道至多对应一个干扰源，且忽略多径传播、建筑或地形遮挡、信号衰落、同频叠加以及天线方向图差异。当上述因素显著时，理想射线和规则扇区可能与实际观测结果产生较大偏差。

(2) 测向误差主要采用确定性区间描述，缺少统计信息。 $\pm 1^\circ$ 误差上界适合构造保守可行域，但不能反映误差分布形态、异常值和检测虚警。只被解释为“无源”或“落

入定向盲区”，模型尚未显式给出漏检概率，因此不能直接回答“在给定置信水平下能否定位”以及“期望漏检率为多少”等问题。

(3) 连续区域计算与离散搜索之间存在精度和复杂度矛盾。当可行域包含圆弧时，最小外接圆及边界候选点需精确处理；若采用网格逼近，则会引入离散化容差，且网格越细、候选点越多，问题二的枚举和问题四的状态更新开销越大。蛇形遍历、2-opt 等启发式方法通常只能保证得到可行解，难以给出与全局最优解之间的差距。

10.3 模型的推广

(1) 由确定性定位推广到概率稳健定位。可将测向误差、接收信号强度、检测概率和虚警率建模为似然函数，利用贝叶斯滤波或粒子滤波更新干扰源位置的后验分布；再以最高后验密度区域或高置信区域替代当前硬约束可行域，并将清除条件写成机会约束或鲁棒约束。

(2) 由单机顺序搜索推广到多机器人协同搜索。可将 S1、S2 扩展为多站协同测向网络，以费舍尔信息矩阵、最小外接圆半径或信息增益为目标选择下一观测位置；同时在路径规划中加入通信连通、避障、续航和多机任务分配约束，采用分布式覆盖或滚动时域优化。对于大范围或高密度干扰源场景，多机协同能够缩短搜索时间并提高定位可靠性。

(3) 由静态二维场景推广到动态三维场景。若干扰源可能移动，可引入速度和加速度状态，将定位问题转化为部分可观测马尔可夫决策过程或在线目标跟踪问题；若部署于无人机、山地、城市楼宇等复杂环境，则需加入高度、非视距传播、地形遮挡和三维天线方向图，用三维体素或三维可行域取代二维扇区，并相应修改交会角和覆盖半径的计算方式。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于程序代码的调试与局部实现、经典几何算法写法的核对以及论文语言润色，详细使用情况见支撑材料。

参考文献

- [1] 修建娟, 何友, 王国宏, 董云龙. 测向交叉定位系统中的交会角研究 [J]. 宇航学报, 2005, 26(03): 282-286.
- [2] 白晶, 王国宏, 王娜, 徐海全. 测向交叉定位系统中的最优交会角研究 [J]. 航空学报, 2009, 30(02): 298-304.
- [3] GuoHong W, BenCai W, You H, et al. 多站无源定位最佳配置分析 [J]. SCIENTIA SINICA Informationis, 2011, 41(10): 1251-1267. DOI:10.1360/zf2011-41-10-1251.

- [4] 曲长文, 王昌海, 冯奇, 等。时差无源定位系统多机编队动态布站优化 [J]. 舰船电子工程, 2017,37 (03):38-43.
- [5] 贺博远, 贺朝会。放射源远程遥控搜寻系统的设计及实验验证 [J]. 核电子学与探测技术, 2015,49 (06):599-601.

附录

```
1
2 from __future__ import annotations
3 import json, math, time, argparse, sys
4 from dataclasses import dataclass, field
5 from typing import List, Tuple, Dict, Any, Optional
6 import numpy as np
7 from urllib.request import Request, urlopen
8 from urllib.error import URLError, HTTPError
9
10 sys.setrecursionlimit(100000) # Welzl
    最小外接圆为递归算法，外圆采样后点数可能上千
11
12 #
    =====
13 # 0. 常量
14 #
    =====
15 BASE_URL = "http://127.0.0.1:2026"
16 ARENA_ID = "default"
17 ROBOT_ID = "202617257031"
18
19 MOVE_SPEED = 5.0 # 机器狗移动速度 m/s
20 MEASURE_TIME = 5.0 # 一次 /measure 虚拟耗时
21 TARGET_RADIUS = 1800.0 # 目标区域半径
22 MIN_RECEIVE_RADIUS = 1000.0 # 全向源最小有效接收半径
23 MAX_RECEIVE_RADIUS = 1500.0 # 最大有效接收半径
24 EPS_DEG = 1.0 # 示向度误差  $\pm 1^\circ$ 
25 NEAR_THRESHOLD = 5.0 # 5m 返回 near
26 CLEAR_RADIUS_MARGIN = 19.0 #
    最小外接圆半径 此值才清除（真实清除半径20m，留1m余量）
27 MAX_MEASURE_PER_SOURCE = 6 #
    单源最多追加测量次数（换侧/补点，均为不重复的有效测量）
28 MAX_SOURCES = 16 #
    干扰源总数10~16，清满16即可确定全部完成
29 POS_GRID = 2.0 #
    同频道测点去重网格(m)：量化到该粒度内视为同一地点
30 VIRTUAL_LIMIT = 359500.0 #
```

```

    虚拟时间硬上限360000，留500s余量
31 MAX_RETRY, RETRY_BACKOFF, HTTP_TIMEOUT = 5, 0.5, 15.0
32
33 # 问题三固定搜索点：5 普查点（原点+四轴点）+ 4
    补漏点（对角线），九点完备覆盖
34 CENSUS_POINTS = [(0.0, 0.0), (900.0, 0.0), (0.0, 900.0),
    (-900.0, 0.0), (0.0, -900.0)]
35 MOPUP_POINTS = [(1000.0, 1000.0), (-1000.0, 1000.0),
36 (-1000.0, -1000.0), (1000.0, -1000.0)]
37
38 # 补漏点顺路调度参数
39 DETOUR_GAIN = 400.0 #
    去源路上绕到补漏点可接受的额外距离(m)
40 MOPUP_FIRST_RATIO = 0.7 #
    补漏点比最近源近到该比例则优先扫
41 MOPUP_DETOUR_LIMIT = 700.0 #
    清除后离未访问补漏点 此值则顺路扫
42
43
44 #
    =====
45 # 1. 几何工具
46 #
    =====
47 def unit(deg: float) -> np.ndarray:
48     """角度(度)转单位向量。"""
49     a = math.radians(deg)
50     return np.array([math.cos(a), math.sin(a)], dtype=float)
51
52
53 def cross2(a: np.ndarray, b: np.ndarray) -> float:
54     return float(a[0] * b[1] - a[1] * b[0])
55
56
57 def two_opt_route(start: np.ndarray, points:
    List[np.ndarray]) -> List[np.ndarray]:
58     """从 start 出发访问全部 points 的近最短路径（最近邻初解 +
    2-opt 局部优化）。"""
59     if len(points) <= 2:
60         return points

```

```

61 remaining, route, cur = list(points), [], start
62 while remaining:
63     k = int(np.argmin([float(np.linalg.norm(p - cur)) for p in
        remaining]))
64     route.append(remaining.pop(k)); cur = route[-1]
65     improved = True
66     while improved:
67         improved = False
68         for i in range(len(route) - 2):
69             for j in range(i + 2, len(route)):
70                 a = start if i == 0 else route[i - 1]
71                 b, c = route[i], route[j]
72                 d = route[j + 1] if j + 1 < len(route) else None
73                 old = float(np.linalg.norm(a - b)) +
                    (float(np.linalg.norm(c - d)) if d is not None else 0.0)
74                 new = float(np.linalg.norm(a - c)) +
                    (float(np.linalg.norm(b - d)) if d is not None else 0.0)
75                 if new + 1e-9 < old:
76                     route[i:j + 1] = reversed(route[i:j + 1]); improved = True
77             return route
78
79
80 def wedge_halfplanes(S: np.ndarray, theta: float, eps:
    float = EPS_DEG):
81     """示向度带 [theta-eps, theta+eps] 转两个半平面 (a,b,c):
        a*x+b*y+c>=0。"""
82     u, v = unit(theta - eps), unit(theta + eps)
83     n1 = np.array([-u[1], u[0]]); h1 = (n1[0], n1[1], -float(n1
        @ S))
84     n2 = np.array([v[1], -v[0]]); h2 = (n2[0], n2[1], -float(n2
        @ S))
85     return [h1, h2]
86
87
88 def clip_halfplane(poly: list, hp) -> list:
89     """Sutherland - Hodgman: 用单个半平面裁剪凸多边形。"""
90     a, b, c = hp
91     if not poly:
92         return []
93     out, n = [], len(poly)

```

```

94     for i in range(n):
95         p, q = poly[i], poly[(i + 1) % n]
96         pin = a * p[0] + b * p[1] + c >= -1e-9
97         qin = a * q[0] + b * q[1] + c >= -1e-9
98         if pin:
99             out.append(p)
100         if pin != qin:
101             den = a * (q[0] - p[0]) + b * (q[1] - p[1])
102             if abs(den) > 1e-12:
103                 t = -(a * p[0] + b * p[1] + c) / den
104                 out.append(p + t * (q - p))
105         return out
106
107
108     def locate_region(points, bearings, eps: float = EPS_DEG,
109                       box_half=2e5):
110         """交会定位区域 =
111             所有示向度带（半平面）的交集凸多边形。返回 (poly,
112             unbounded)，空则 poly=None。"""
113         if len(points) != len(bearings) or len(points) < 2:
114             raise ValueError("需要至少2个检测点且数量一致")
115         ctr = np.mean([np.asarray(p, dtype=float) for p in points],
116                       axis=0)
117         L = box_half
118         poly = [ctr + np.array([-L, -L]), ctr + np.array([L, -L]),
119                 ctr + np.array([L, L]), ctr + np.array([-L, L])]
120         for S, th in zip(points, bearings):
121             for hp in wedge_halfplanes(np.asarray(S, dtype=float),
122                                       float(th), eps):
123                 poly = clip_halfplane(poly, hp)
124             if not poly:
125                 return None, False
126         unbounded = max(float(np.abs(v).max()) for v in poly) > 1e4
127         return poly, unbounded
128
129
130     def convex_diameter(poly: list):
131         """旋转卡壳求凸多边形直径，返回 (D, i, j)。"""
132         n = len(poly)
133         if n < 2:

```

```

129     return 0.0, 0, 0
130     if n == 2:
131         return float(np.linalg.norm(poly[1] - poly[0])), 0, 1
132     j, best, bi, bj = 1, 0.0, 0, 0
133     for i in range(n):
134         ni = (i + 1) % n
135         while True:
136             nj = (j + 1) % n
137             if cross2(poly[ni] - poly[i], poly[nj] - poly[j]) > 1e-12:
138                 j = nj
139             else:
140                 break
141             d = float(np.linalg.norm(poly[j] - poly[i]))
142             if d > best:
143                 best, bi, bj = d, i, j
144         return best, bi, bj
145
146
147     def welzl_min_enclosing_circle(points) -> Tuple[np.ndarray,
148         float]:
149         """Welzl 随机增量法求点集最小外接圆，返回 (center,
150             radius)"""
151         pts = [np.asarray(p, dtype=float) for p in points]
152         n = len(pts)
153         if n == 0:
154             return np.zeros(2), 0.0
155         if n == 1:
156             return pts[0].copy(), 0.0
157
158         def c2(a, b):
159             return (a + b) / 2.0, float(np.linalg.norm(a - b) / 2.0)
160
161         def c3(a, b, c):
162             ax, ay = a; bx, by = b; cx, cy = c
163             d = 2.0 * (ax * (by - cy) + bx * (cy - ay) + cx * (ay - by))
164             if abs(d) < 1e-12: #
165                 三点共线，退化为最长边外接圆
166                 pairs = [(a, b), (b, c), (a, c)]
167                 return max((c2(x, y) for x, y in pairs), key=lambda t: t[1])
168             ux = ((ax**2 + ay**2) * (by - cy) + (bx**2 + by**2) * (cy -

```

```

    ay) + (cx**2 + cy**2) * (ay - by)) / d
166 uy = ((ax**2 + ay**2) * (cx - bx) + (bx**2 + by**2) * (ax -
    cx) + (cx**2 + cy**2) * (bx - ax)) / d
167 cc = np.array([ux, uy])
168 return cc, float(np.linalg.norm(a - cc))
169
170 def welzl(P, R, k):
171     if k == 0 or len(R) == 3:
172         if len(R) == 0: return np.zeros(2), 0.0
173         if len(R) == 1: return R[0].copy(), 0.0
174         if len(R) == 2: return c2(R[0], R[1])
175         return c3(R[0], R[1], R[2])
176     p = P[k - 1]
177     cc, r = welzl(P, R, k - 1)
178     if float(np.linalg.norm(p - cc)) <= r + 1e-9:
179         return cc, r
180     return welzl(P, R + [p], k - 1)
181
182 perm = np.random.default_rng(42).permutation(n)
183 return welzl([pts[i] for i in perm], [], n)
184
185
186 #
187 # =====
188 # 2. 问题二 闭式第二检测点 / 迭代定位
189 #
190 # =====
191
192 def ray_circle_interval(s: np.ndarray, u: np.ndarray,
193     radius: float = TARGET_RADIUS):
194     """射线 s+t*u (t 0) 与目标圆盘的参数区间
195     [t_lo,t_hi], 无交集返回None。"""
196     B = 2.0 * float(s @ u)
197     C = float(s @ s - radius ** 2)
198     disc = B * B - 4.0 * C
199     if disc < 0:
200         return None
201     root = math.sqrt(max(0.0, disc))
202     t1, t2 = (-B - root) / 2.0, (-B + root) / 2.0
203     lo, hi = max(0.0, min(t1, t2)), max(t1, t2)
204     return (lo, hi) if hi >= lo else None

```

```

200
201
202 def q2_closed_form(S1, theta,
203                     guaranteed_r=MIN_RECEIVE_RADIUS,
204                     near=NEAR_THRESHOLD, max_r=MAX_RECEIVE_RADIUS,
205                     target_r=TARGET_RADIUS):
206     """问题二闭式第二检测点:  $S2=S1+a*u\pm b*n$ ,  $b=\sqrt{R^2-h^2}$ 。
207     保证第一次示向射线上的可能源位置都落在第二点 R
208     邻域内, 且横向交会基线尽量大。"""
209     u = unit(theta)
210     iv = ray_circle_interval(np.asarray(S1, dtype=float), u,
211                             target_r)
212     if iv is None:
213         raise ValueError("示向射线与目标区域无交集")
214     t0, t1 = max(near, iv[0]), min(max_r, iv[1])
215     if t1 <= t0:
216         raise ValueError("第一次可能源区间为空")
217     n = np.array([-u[1], u[0]])
218     h, a = (t1 - t0) / 2.0, (t0 + t1) / 2.0
219     b = 0.0 if h > guaranteed_r else math.sqrt(max(0.0,
220                                                     guaranteed_r ** 2 - h * h))
221     mid = np.asarray(S1, dtype=float) + a * u
222     return {"candidates": np.vstack([mid + b * n, mid - b * n])}
223
224
225 def _seg_disk_intersections(v: np.ndarray, w: np.ndarray,
226                             N: np.ndarray, r: float):
227     """线段  $v \rightarrow w$  与圆盘  $(N, r)$  的交点 (参数  $t \in [0, 1]$ )。"""
228     d = w - v; f = v - N
229     a = float(d @ d)
230     out = []
231     if a < 1e-12:
232         return out
233     b = 2.0 * float(f @ d); c = float(f @ f - r * r)
234     disc = b * b - 4.0 * a * c
235     if disc < 0:
236         return out
237     root = math.sqrt(disc)
238     for t in ((-b - root) / (2.0 * a), (-b + root) / (2.0 * a)):
239         if -1e-9 <= t <= 1.0 + 1e-9:

```

```

234 out.append(v + min(1.0, max(0.0, t)) * d)
235 return out
236
237
238 def _point_in_convex(p, poly, tol=1e-6) -> bool:
239     """点是否在有序凸多边形内（顺/逆时针自适应）。"""
240     if len(poly) < 3:
241         return False
242     pos = neg = False
243     m = len(poly)
244     for i in range(m):
245         a, b = poly[i], poly[(i + 1) % m]
246         cr = (b[0] - a[0]) * (p[1] - a[1]) - (b[1] - a[1]) * (p[0]
            - a[0])
247         if cr > tol:
248             pos = True
249         elif cr < -tol:
250             neg = True
251         if pos and neg:
252             return False
253     return True
254
255
256 def _disk_disk_intersections(A: np.ndarray, B: np.ndarray,
    r: float):
257     """两个等半径 r 圆盘边界的交点。"""
258     d = B - A
259     dist = float(np.linalg.norm(d))
260     if dist < 1e-9 or dist > 2.0 * r + 1e-9:
261         return []
262     a = dist / 2.0
263     h2 = r * r - a * a
264     if h2 < 0:
265         return []
266     h = math.sqrt(h2)
267     mid = (A + B) / 2.0
268     off = h * np.array([-d[1] / dist, d[0] / dist])
269     return [mid + off, mid - off]
270
271

```

```

272 def apply_outside_disks(poly, disks, r: float =
      MIN_RECEIVE_RADIUS, circle_steps: int = 720):
273     """定位凸多边形 poly 与若干“圆盘补集” $\{X: ||X-N|| > r\}$ 
      的交——挖掉每个 no_signal 点的 r 邻域。
274     问题三为全向源、接收半径 1000，已确认存在的频道在 N 处
      no_signal 必满足  $||G-N|| > 1000$ ，
275     故挖掉  $||X-N|| > 1000$  是严格成立的约束。
276     挖洞后非凸，其凸包极点有四类，缺一不可（漏类会让 MEC 偏小、误判收敛）：
277     原多边形中在所有圆盘外的顶点；
278     多边形边与圆盘的交点；
279     “暴露圆弧”上的点——圆盘削掉多边形一角时，两交点间暴露给可行域的圆弧向外
280     按 circle_steps 密采样（ $0.5^\circ$  步长、 $r=1000$ 
      时矢高 0.01m，远小于 19m 阈值与 1m 裕量）；
281     两个圆盘边界的交点（多圆交互削角处亦可能成为凸包极点）。"""
282     if not disks or poly is None or len(poly) < 3:
283         return poly
284     P = [np.asarray(p, float) for p in poly]
285     Ns = [np.asarray(d, float) for d in disks]
286
287     def out_others(p, skip):
288         for k in range(len(Ns)):
289             if k == skip:
290                 continue
291             if float(np.linalg.norm(p - Ns[k])) < r - 1e-9:
292                 return False
293             return True
294
295     pts: List[np.ndarray] = []
296     m = len(P)
297     for v in P:                                     #
298         原顶点（须在所有圆外）
299         if all(float(np.linalg.norm(v - Nc)) >= r - 1e-9 for Nc in
      Ns):
300             pts.append(v)
301             for i in range(m):                       #
302                 边-圆交点
303                 v, w = P[i], P[(i + 1) % m]
304                 for k, Nc in enumerate(Ns):
305                     for q in _seg_disk_intersections(v, w, Nc, r):
306                         if out_others(q, k):

```

```

305 pts.append(q)
306 for k, Nc in enumerate(Ns):                                     #
    暴露圆弧密采样
307 for s in range(circle_steps):
308 a = 2.0 * math.pi * s / circle_steps
309 q = Nc + r * np.array([math.cos(a), math.sin(a)])
310 if _point_in_convex(q, P) and out_others(q, k):
311 pts.append(q)
312 for i in range(len(Ns)):                                       #
    圆-圆交点（须在P内、对其他圆在外）
313 for j in range(i + 1, len(Ns)):
314 for q in _disk_disk_intersections(Ns[i], Ns[j], r):
315 if _point_in_convex(q, P) and out_others(q, -1):
316 pts.append(q)
317 return pts if pts else list(poly)                             #
    数值兜底：可行域不应为空，空则退回原多边形（保守不收敛）
318
319
320 def iterative_localize(observations, no_sig=(),
    clear_r=CLEAR_RADIUS_MARGIN,
321 exclude_r=MIN_RECEIVE_RADIUS):
322 """由若干（检测点，示向度）观测求定位区域及其最小外接圆。
323 no_sig: 该频道历次 no_signal 的位置；问题三可据此挖掉各自的
    1000m 圆盘以收紧区域。
324 返回 (poly, center, radius, can_clear, unbounded) (poly
    仍为未挖洞的示向带交，供补点用)。"""
325 stations = [o[0] for o in observations]
326 bearings = [o[1] for o in observations]
327 poly, unb = locate_region(stations, bearings)
328 if poly is None or len(poly) < 3 or unb:
329 return poly, None, float("inf"), False, unb
330 pts = apply_outside_disks(poly, list(no_sig), exclude_r) if
    no_sig else poly
331 center, radius = welzl_min_enclosing_circle(np.array(pts))
332 return poly, center, radius, radius <= clear_r, unb
333
334
335 #
    =====
336 # 3. 频道状态

```

```

337 #
338
339 @dataclass
340 class ChannelState:
341     channel: int
342     status: str = "unknown"          # unknown / detected /
343     near / cleared
344     observations: List[Tuple[Tuple[float, float], float]] =
345         field(default_factory=list)
346     no_signal_positions: List[Tuple[float, float]] =
347         field(default_factory=list)
348     tried_keys: set = field(default_factory=set)    #
349     该频道已成功执行measure的量化坐标，禁止重复
350     clear_attempts: int = 0
351
352 #
353
354 # 4. 模拟器 HTTP 客户端
355 #
356
357 class RobotClient:
358     def __init__(self, base_url=BASE_URL, robot_id=ROBOT_ID):
359         self.base_url, self.robot_id = base_url, robot_id
360         self.counter = 0
361         self.log: List[Dict[str, Any]] = []
362
363     def _next_id(self, prefix):
364         self.counter += 1
365         return f"{prefix}-{self.counter}"
366
367     def _post(self, path, payload):
368         """POST: 新动作新 request_id; 网络断连在重试中复用同一
369             payload (含同一 request_id)。"""
370         data = json.dumps(payload).encode("utf-8")
371         last = None
372         for attempt in range(MAX_RETRY):
373             try:
374                 req = Request(self.base_url + path, data=data,
375                     headers={"Content-Type": "application/json"}, method="POST")

```

```

369 with urlopen(req, timeout=HTTP_TIMEOUT) as resp:
370     result = json.loads(resp.read().decode("utf-8"))
371     self.log.append({"path": path, "request": payload,
372                     "response": result, "time": time.time()})
373 return result
374 except HTTPError as e:
375     # 格式/冲突类错误, 重试无意义
376     print(f"[HTTP ERROR] {path} -> {e.code} {e.reason}")
377     return None
378 except (URLError, TimeoutError, ConnectionError, OSError)
379     as e: # 断连, 复用原ID重试
380     last = e
381     if attempt < MAX_RETRY - 1:
382         w = RETRY_BACKOFF * 2 ** attempt
383         print(f"[网络重试] {path} 第{attempt+1}次失败:
384             {e}, {w:.1f}s后重试")
385         time.sleep(w)
386     except json.JSONDecodeError:
387         return None
388     print(f"[ERROR] {path} 重试{MAX_RETRY}次仍失败: {last}")
389     return None
390
391 def enter(self):
392     return self._post("/enter", {"arena_id": ARENA_ID,
393                                 "robot_id": self.robot_id,
394                                 "request_id": self._next_id("enter")})
395
396 def exit(self):
397     return self._post("/exit", {"arena_id": ARENA_ID,
398                                 "robot_id": self.robot_id,
399                                 "request_id": self._next_id("exit")})
400
401 def measure(self, pos, channel):
402     return self._post("/measure", {
403         "arena_id": ARENA_ID, "robot_id": self.robot_id,
404         "request_id": self._next_id("measure"),
405         "position": {"x": float(pos[0]), "y": float(pos[1])},
406         "channel": int(channel)})
407
408 def clear(self, pos, channel):

```

```

402     return self._post("/clear", {
403         "arena_id": ARENA_ID, "robot_id": self.robot_id,
404         "request_id": self._next_id("clear"),
405         "position": {"x": float(pos[0]), "y": float(pos[1])},
406         "channel": int(channel)})
407
408     #
409     # =====
410     # 5. 问题三求解器
411     # =====
412
413     class Q3Solver:
414     def __init__(self, client):
415         self.client = client
416         self.census_points = [np.array(p, dtype=float) for p in
417             CENSUS_POINTS]
418         self.mopup_pts = [np.array(p, dtype=float) for p in
419             MOPUP_POINTS]
420         self.channels: Dict[int, ChannelState] = {c:
421             ChannelState(c) for c in range(1, 21)}
422         self.cleared_count = 0
423         self.real_start = 0.0
424         self.real_budget = 1200.0
425         self.virtual_time = 0.0
426         self.current_pos = np.zeros(2)
427         self.mopup_visited = set()
428         self.inplace_failed = set() #
429         普查途中就地清除未中、需交回追踪补测的频道
430         self.use_nosig_exclude = True # 是否用 no_signal
431         的“距源>1000”外圆约束收紧定位（问题三严格成立）
432         self.schedule_mode = "nearest" # 追踪调度: nearest=最近源
433         / gain=信息增益代价比（A/B用）
434
435     def _nosig(self, st):
436     """本频道可用于外圆排除的 no_signal
437         位置（开关关闭时返回空，便于A/B对照）。"""
438     return st.no_signal_positions if self.use_nosig_exclude
439         else ()
440

```

```

431 # ----- 基础 -----
432 def time_ok(self, reserve=30.0) -> bool:
433     if time.time() - self.real_start > self.real_budget -
         reserve:
434     return False
435     return self.virtual_time <= VIRTUAL_LIMIT
436
437 def _pkey(self, pos):
438     """同频道测点去重键：坐标量化到 POS_GRID 网格。"""
439     return (round(float(pos[0]) / POS_GRID),
         round(float(pos[1]) / POS_GRID))
440
441 def _is_tried(self, ch, pos) -> bool:
442     return self._pkey(pos) in self.channels[ch].tried_keys
443
444 def _first_unused(self, ch, cand, legal=None):
445     """按候选顺序返回第一个没测过且合法的点；都测过返回None。"""
446     for p in cand:
447         p = np.asarray(p, dtype=float)
448         if not np.all(np.isfinite(p)) or self._is_tried(ch, p):
449             continue
450         if legal is not None and not legal(p):
451             continue
452         return p
453     return None
454
455 @staticmethod
456 def _legal_track_point(p, margin=200.0):
457     #
         追踪补点不跑出目标圆太远（机器狗虽可去圆外，但追踪无需远走）
458     return float(np.linalg.norm(p)) <= TARGET_RADIUS + margin
459
460 def _measure(self, pos, ch):
461     resp = self.client.measure(pos, ch)
462     if resp is None:
463         return None
464     if resp.get("accepted") is not True:
465         return "rejected"
466     self.virtual_time = float(resp.get("virtual_time_s",
         self.virtual_time))

```

```

467     self.current_pos = np.asarray(pos, dtype=float)
468     return resp.get("measure_result")
469
470     def _clear(self, pos, ch):
471         resp = self.client.clear(pos, ch)
472         if resp is None:
473             return None
474         if resp.get("accepted") is not True:
475             return "rejected"
476         self.virtual_time = float(resp.get("virtual_time_s",
477                                         self.virtual_time))
478         self.current_pos = np.asarray(pos, dtype=float)
479         return resp.get("clear_result")
480
481     def _svd(self):
482         return self.client.log[-1]["response"].get("svd_deg") if
483             self.client.log else None
484
485     def _absorb(self, ch, pos, result, tag=""):
486         """统一吸收一次测量结果（登记已测坐标，避免重复）。"""
487         st = self.channels[ch]
488         gp = (float(pos[0]), float(pos[1]))
489         st.tried_keys.add(self._pkey(gp))
490         if result == "direction":
491             svd = self._svd()
492             if svd is not None:
493                 st.observations.append((gp, float(svd)))
494                 st.status = "detected"
495             elif result == "near":
496                 st.status = "near"
497             if self._clear(gp, ch) == "success":
498                 st.status = "cleared"; self.cleared_count += 1
499                 print(f"    频道{ch} near→清除成功")
500             elif result == "no_signal":
501                 st.no_signal_positions.append(gp)
502             return st.status
503
504     # ----- 阶段1: 普查（动态最近邻 +
505         途中就近清除已收敛源） -----
506     def _clear_ready_in_place(self):

```

```

504 """普查点扫完后：对已被 2 点交会、定位区域最小外接圆 19m
    的源就近清除。
505 只做 clear
    这一早晚要发生的终态动作以省折返，绝不提前补点、不打乱后续全局追踪调度；
506 清除未中则登记交回追踪阶段，保证不漏。"""
507 while self.time_ok():
508     best = None
509     for ch, st in self.channels.items():
510         if st.status != "detected" or len(st.observations) < 2 or
            ch in self.inplace_failed:
511             continue
512         _, center, radius, can_clear, unb =
            iterative_localize(st.observations, self._nosig(st))
513         if can_clear and center is not None and not unb:
514             d = float(np.linalg.norm(center - self.current_pos))
515             if best is None or d < best[0]:
516                 best = (d, ch, center, radius)
517             if best is None:
518                 return
519         _, ch, center, radius = best
520         st = self.channels[ch]
521         st.clear_attempts += 1
522         if self._clear(center, ch) == "success":
523             st.status = "cleared"; self.cleared_count += 1
524             print(f"    频道{ch}
                普查途中就近清除（定位半径{radius:.1f}m）")
525         if self.cleared_count >= MAX_SOURCES:
526             return
527         else:
528             self.inplace_failed.add(ch) #
                估计偏出清除圆：交回追踪补测，本阶段不再碰
529         return
530
531 def census(self):
532     print("[阶段1] 多位置频道普查（途中就近清除已收敛源）...")
533     remaining = [np.array(p, dtype=float) for p in
                    self.census_points]
534     while remaining:
535         if not self.time_ok():
536             return

```

```

537 #
    动态选离当前位置最近的未访问普查点（就近清除会偏离固定路线，动态重选更合理）
538 k = int(np.argmin([float(np.linalg.norm(p -
    self.current_pos)) for p in remaining]))
539 gp_arr = remaining.pop(k)
540 gp = (float(gp_arr[0]), float(gp_arr[1]))
541 print(f"  普查位置({gp[0]:.0f},{gp[1]:.0f})...")
542 for ch in range(1, 21):
543     st = self.channels[ch]
544     if st.status == "cleared":
545         continue
546 #
    已detected也继续测：被多个普查点看到即可直接交会，省第二点移动
547 if not self.time_ok():
548     return
549 self._absorb(ch, gp, self._measure(gp, ch))
550 self._clear_ready_in_place()
551 if self.cleared_count >= MAX_SOURCES:
552     return
553 multi = [c for c, s in self.channels.items() if s.status ==
    "detected" and len(s.observations) >= 2]
554 found = [c for c, s in self.channels.items() if s.status in
    ("detected", "near")]
555 print(f"  普查完成：发现{len(found)}个
    {found}，其中{len(multi)}个多次观测可直接定位 {multi}")
556
557 # ----- 阶段2：追踪 + 补漏顺路统一调度 -----
558 def _ray_midpoint(self, S1, theta):
559     """单次观测：源在示向射线与目标圆交段上，取该段中点作位置估计（用于路径调度）
560     S1 = np.asarray(S1, dtype=float); u = unit(theta)
561     iv = ray_circle_interval(S1, u, TARGET_RADIUS)
562     if iv is None:
563         return S1 + 900.0 * u
564     t0 = max(NEAR_THRESHOLD, iv[0]); t1 =
        min(MAX_RECEIVE_RADIUS, iv[1])
565     t = 0.5 * (t0 + t1) if t1 > t0 else 900.0
566     return S1 + t * u
567
568 def _est_pos(self, ch):
569     """待清源的估计位置（用于最近邻调度）： 2次观测取最小外接圆圆心，否则取射线

```

```

570     st = self.channels[ch]
571     if len(st.observations) >= 2:
572         poly, unb = locate_region([o[0] for o in st.observations],
573                                   [o[1] for o in st.observations])
574         if poly is not None and len(poly) >= 3 and not unb:
575             pts = apply_outside_disks(poly, self._nosig(st)) if
                    self.use_nosig_exclude else poly
576             c, _ = welzl_min_enclosing_circle(np.array(pts))
577             return c
578         if st.observations:
579             return self._ray_midpoint(*st.observations[0])
580         return np.zeros(2)
581
582     def track_and_clear(self):
583         print("[阶段2] 逐源追踪清除（补漏点顺路统一调度）...")
584         while True:
585             if not self.time_ok():
586                 print(" 时间不足，追踪中断"); break
587             if self.cleared_count >= MAX_SOURCES:          #
                    已达总数上限，确定无剩余源
588             print(f"
                    已清除{MAX_SOURCES}个（题目上限），确定全部完成，提前结束");
                    break
589         pending = [c for c, s in self.channels.items()]
590         if s.status in ("detected", "near")
591         and (s.clear_attempts == 0 or c in self.inplace_failed)]
592         unvisited = [(i, mp) for i, mp in enumerate(self.mopup_pts)
                    if i not in self.mopup_visited]
593         if not pending and not unvisited:
594             break
595
596         # 最近待清除源 / 信息增益-代价比调度
597         if pending:
598             if self.schedule_mode == "gain":
599                 def _prio(c):
600                     d = float(np.linalg.norm(self._est_pos(c) -
                    self.current_pos))
601                 st = self.channels[c]
602                 unc = 1500.0
603                 if len(st.observations) >= 2:

```

```

604     _, _, rad, _, _ = iterative_localize(st.observations,
        self._nosig(st))
605     if np.isfinite(rad):
606         unc = rad
607         return -(unc / (d / MOVE_SPEED + MEASURE_TIME))    #
            单位代价信息增益大者优先
608     pending.sort(key=_prio)
609     else:
610     pending.sort(key=lambda c:
        float(np.linalg.norm(self._est_pos(c) -
            self.current_pos)))
611     nch, target = pending[0], self._est_pos(pending[0])
612     self.inplace_failed.discard(nch)
613     d_src = float(np.linalg.norm(target - self.current_pos))
614     else:
615     nch, target, d_src = None, None, float("inf")
616
617     # 绕路增量最小的补漏点
618     best = None
619     for i, mp in unvisited:
620     d_mp = float(np.linalg.norm(mp - self.current_pos))
621     detour = d_mp + float(np.linalg.norm(target - mp)) - d_src
        if target is not None else d_mp
622     if best is None or detour < best[0]:
623     best = (detour, i, mp, d_mp)
624
625     go_mopup = best is not None and (
626     target is None or best[0] <= DETOUR_GAIN or best[3] <=
        d_src * MOPUP_FIRST_RATIO)
627     if go_mopup:
628     _, i, mp, _ = best
629     print(f"    [顺路补漏]
        先扫({mp[0]:.0f},{mp[1]:.0f}), 绕路增量{best[0]:.0f}m")
630     self.scan_mopup(i, mp)
631     continue
632     if nch is None:
633     break
634
635     print(f"
        处理频道{nch} (已有{len(self.channels[nch].observations)}次观测) ...")

```

```

636 self.process_channel(nch)
637 if self.channels[nch].status == "cleared":
638     self._detour_mopup_if_near()
639
640 def _detour_mopup_if_near(self):
641     """清除一个源后：离未访问补漏点足够近则顺路扫。"""
642     for i, mp in enumerate(self.mopup_pts):
643         if i not in self.mopup_visited and float(np.linalg.norm(mp
644             - self.current_pos)) <= MOPUP_DETOUR_LIMIT:
645             self.scan_mopup(i, mp)
646
647 def scan_mopup(self, idx, mp):
648     """在一个补漏点扫描所有未发现/未清除频道，新发现的源立即追踪清除。"""
649     self.mopup_visited.add(idx)
650     gp = (float(mp[0]), float(mp[1]))
651     new_found = []
652     for ch in range(1, 21):
653         st = self.channels[ch]
654         if st.status in ("cleared", "detected", "near"):
655             continue
656         if not self.time_ok():
657             break
658         before = st.status
659         self._absorb(ch, gp, self._measure(gp, ch), tag="mopup")
660         if st.status == "detected" and before != "detected":
661             new_found.append(ch)
662             print(f"  补漏发现频道{ch} ({gp[0]:.0f},{gp[1]:.0f}) ")
663         for ch in new_found:
664             if self.time_ok() and self.channels[ch].status ==
665                 "detected":
666                 self.process_channel(ch)
667
668 # ----- 单源定位的候选测点 -----
669 def _single_obs_candidates(self, st):
670     """仅1次观测时的补点序列（全向源）：
671     Q2两个对称点（离当前近者优先）→ 沿示向朝源逼近 → 法向兜底。
672     几何上第一个Q2点若no_signal，源必更靠近另一对称点，换另一侧即有信号，无需隔
673     pos0, svd0 = st.observations[0]
674     S1 = np.asarray(pos0, float); u = unit(svd0); n =
675         np.array([-u[1], u[0]])

```

```

673     cands = []
674     try:
675         q = list(q2_closed_form(pos0, svd0)["candidates"])
676         q.sort(key=lambda p: float(np.linalg.norm(p -
677             self.current_pos)))
677         cands.extend(q)
678     except Exception:
679         pass
680     mid = self._ray_midpoint(pos0, svd0)
681     for frac in (0.55, 0.8, 1.05):           #
682         沿示向朝源逼近（换侧仍失败的兜底）
683         cands.append(S1 + frac * (mid - S1))
684     for s in (1.0, -1.0):                   #
685         法向更远两侧
686         cands.append(S1 + s * 1200.0 * n)
687     return cands
688
689     def _pick_third_unused(self, ch, st, poly):
690         """ 2 观测但区域偏大：沿定位直径法向补点，跳过已测坐标；候选耗尽返回 None。 """
691         _, i, j = convex_diameter(poly)
692         vi, vj = poly[i], poly[j]
693         ddir = (vj - vi) / max(float(np.linalg.norm(vj - vi)), 1e-9)
694         perp = np.array([-ddir[1], ddir[0]])
695         center = (vi + vj) / 2.0
696         cands = []
697         for dist in (600.0, 800.0, 1000.0, 1200.0):
698             for s in (1.0, -1.0):
699                 cands.append(center + s * dist * perp)
700             for s in (1.0, -1.0):
701                 cands.append(center + s * 700.0 * ddir)
702         cands.sort(key=lambda p: float(np.linalg.norm(p -
703             self.current_pos)))
704         return self._first_unused(ch, cands,
705             self._legal_track_point)
706
707     def _fallback_unused(self, ch, st):
708         """ 交会退化/无界时的补点，跳过已测坐标。 """
709         pos, svd = st.observations[-1]
710         S = np.asarray(pos, float); u = unit(svd); n =
711             np.array([-u[1], u[0]])

```

```

707     cands = []
708     for s in (1.0, -1.0):
709         for dist in (600.0, 1000.0, 1400.0):
710             cands.append(S + s * dist * n)
711             cands += [S + 600.0 * u, S - 400.0 * u]
712             cands.sort(key=lambda p: float(np.linalg.norm(p -
                    self.current_pos)))
713     return self._first_unused(ch, cands,
                    self._legal_track_point)
714
715     def process_channel(self, ch):
716         """单源定位清除（全向）。每次补测都选未测过的点、严格不重复：
717         1观测→Q2对称点(无信号自动换另一侧)→沿示向逼近； 2观测→交会，区域不够小
718         就沿直径法向补新点，直到最小外接圆 19m再清除。"""
719         st = self.channels[ch]
720         for _ in range(MAX_MEASURE_PER_SOURCE):
721             if not self.time_ok():
722                 return
723             if len(st.observations) == 1:                                     #
724                 仅1次观测：取第一个未测候选
725             npt = self._first_unused(ch,
                    self._single_obs_candidates(st),
526             self._legal_track_point)
727             if npt is None:
728                 break
729             self._absorb(ch, npt, self._measure(npt, ch))
730             if st.status == "cleared":
731                 return
732             continue
733
734             poly, center, radius, can_clear, unb =
                    iterative_localize(st.observations, self._nosig(st))
735             if poly is None or len(poly) < 3 or unb:                                     #
736                 退化/无界：换未测过的点补
737             npt = self._fallback_unused(ch, st)
738             if npt is None:
739                 break
740             self._absorb(ch, npt, self._measure(npt, ch))
741             if st.status == "cleared":
742                 return

```

```

741     continue
742     if can_clear:                                     #
        区域足够小：清除
743     st.clear_attempts += 1
744     if self._clear(center, ch) == "success":
745     st.status = "cleared"; self.cleared_count += 1
746     print(f"    频道{ch} 清除成功（定位半径{radius:.1f}m）");
        return
747     npt = self._pick_third_unused(ch, st, poly)       #
        清除失败/区域偏大：补未测新点
748     if npt is None:
749     break
750     self._absorb(ch, npt, self._measure(npt, ch))
751     if st.status == "cleared":
752     return
753
754     if len(st.observations) >= 2:                     #
        兜底：用最新估计最后清一次
755     _, center, _, _, _ = iterative_localize(st.observations,
        self._nosig(st))
756     if center is not None and self._clear(center, ch) ==
        "success":
757     st.status = "cleared"; self.cleared_count += 1
758     print(f"    频道{ch} 最后尝试清除成功"); return
759     print(f"    频道{ch}
        未能清除（观测{len(st.observations)}次）")
760
761     # ----- 阶段3：补漏兜底 -----
762     def mop_up(self):
763     if self.cleared_count >= MAX_SOURCES:
764     print("[阶段3] 已清满16个（题目上限），跳过补漏"); return
765     remain = [(i, mp) for i, mp in enumerate(self.mopup_pts) if
        i not in self.mopup_visited]
766     if not remain:
767     print("[阶段3] 补漏点已在追踪时全部顺路覆盖，跳过"); return
768     print(f"[阶段3]
        {len(remain)}个补漏点未顺路覆盖，兜底检测...")
769     pts = [mp for _, mp in remain]
770     route = two_opt_route(self.current_pos, pts)
771     idx_of = {(round(float(mp[0])), round(float(mp[1]))): i for

```

```

        i, mp in remain}
772 while route:
773     if not self.time_ok():
774         break
775     route.sort(key=lambda p: float(np.linalg.norm(p -
        self.current_pos)))
776     mp = route.pop(0)
777     self.scan_mopup(idx_of[(round(float(mp[0])),
        round(float(mp[1])))], mp)

778
779 # ----- 主流程 -----
780 def run(self, save_log="robot_log_q3.json"):
781     enter = self.client.enter()
782     if enter is None or enter.get("accepted") is not True:
783         print("[ERROR] /enter
        失败（确认模拟器已进入问题三测试并过5秒倒计时）"); return
784     self.real_budget =
        float(enter.get("remaining_real_duration_s", 1200.0))
785     self.real_start = time.time()
786     print(f"[INFO] 问题三
        可用现实时间={self.real_budget:.0f}s")
787     self.census() #
        阶段1: 5点普查
788     self.track_and_clear() #
        阶段2: 逐源追踪+补漏顺路
789     if self.time_ok(reserve=15):
790         self.mop_up() #
        阶段3: 补漏兜底
791     print(f"[INFO] 任务结束, 共清除 {self.cleared_count}
        个干扰源, 虚拟时间 {self.virtual_time:.0f}s")
792     self.client.exit()
793     if save_log:
794         with open(save_log, "w", encoding="utf-8") as f:
795             json.dump(self.client.log, f, ensure_ascii=False, indent=2)
796     print(f"[INFO] 行为日志已保存到
        {save_log} ({len(self.client.log)}条)")

797
798
799 #
        =====

```

```

800 # 6. 入口
801 #
=====
802 def main():
803     ap = argparse.ArgumentParser(description="CUMCM-B 问题三
      全向干扰源自动定位清除（独立正式版）")
804     ap.add_argument("--robot_id", default=ROBOT_ID)
805     ap.add_argument("--base_url", default=BASE_URL)
806     ap.add_argument("--log", default="robot_log_q3.json")
807     a = ap.parse_args()
808     client = RobotClient(base_url=a.base_url,
      robot_id=str(a.robot_id))
809     Q3Solver(client).run(save_log=a.log)
810
811
812 if __name__ == "__main__":
813     main()

```

Listing 1: 问题三机械狗代码：全向无线电干扰源快速自动定位与清除（solution_q3.py）

```

1
2 from __future__ import annotations
3 import json, math, time, argparse
4 from dataclasses import dataclass, field
5 from typing import List, Tuple, Dict, Any
6 import numpy as np
7 from urllib.request import Request, urlopen
8 from urllib.error import URLError, HTTPError
9
10 #
=====
11 # 0. 常量（全部对齐题目与附件1/2）
12 #
=====
13 BASE_URL = "http://127.0.0.1:2026"
14 ARENA_ID = "default"
15 ROBOT_ID = "202617257031"
16
17 MOVE_SPEED = 5.0 # 机器狗移动速度 m/s
18 MEASURE_TIME = 5.0 # 一次 /measure 虚拟耗时
19 TARGET_RADIUS = 1800.0 # 目标区域半径

```

```

20 MIN_RECEIVE_RADIUS = 1000.0      # 最小有效接收半径
21 MAX_RECEIVE_RADIUS = 1500.0      # 最大有效接收半径
22 EPS_DEG = 1.0                    # 示向度误差 ±1°
23 NEAR_THRESHOLD = 5.0              # 5m 返回 near
24 CLEAR_RADIUS_MARGIN = 19.0        #
    最小外接圆半径 此值才清除（真实清除半径20m，留1m余量）
25 Q4_MEASURE_CAP = 16              #
    单源追加测量上限：定向源需左右交错试探半平面内侧，候选更多
26 MAX_SOURCES = 16                 #
    干扰源总数10~16，清满16即可确定全部完成
27 POS_GRID = 2.0                   # 同频道测点去重网格(m)
28 VIRTUAL_LIMIT = 359500.0         #
    虚拟时间硬上限360000，留500s余量
29 MAX_RETRY, RETRY_BACKOFF, HTTP_TIMEOUT = 5, 0.5, 15.0
30 INPLACE_CLEAR_MAXD = 700.0       #
    发现网途中只就近清除距当前 700m的收敛源（过远偏离巡回反而慢）
31
32 # ---- 27点发现网（三角格；step=15m、45369网格离散核验
    worst span=202.8°、裕量22.8°） ----
33 Q4_DISCOVERY_POINTS = [
34     (-300,-202),(200,664),(700,-202),(-800,664),(200,-1068),(-1300,-202),
35     (-800,-1068),(1200,664),(-300,1530),(1200,-1068),(700,1530),(1700,-202),
36     (-1800,664),(-300,-1934),(-1300,1530),(700,-1934),(-1800,-1068),(1700,1530),
37     (2200,664),(-2300,-202),(-1300,-1934),(200,2396),(2200,-1068),(-800,2396),
38     (1700,-1934),(1200,2396),(2700,-202)]
39
40
41 #
    =====
42 # 1. 几何工具
43 #
    =====
44 def unit(deg: float) -> np.ndarray:
45     a = math.radians(deg)
46     return np.array([math.cos(a), math.sin(a)], dtype=float)
47
48
49 def two_opt_route(start: np.ndarray, points:
    List[np.ndarray]) -> List[np.ndarray]:
50     """从 start 出发访问全部 points 的近最短路径（最近邻初解 +

```

```

    2-opt 局部优化)。"""
51 if len(points) <= 2:
52     return points
53     remaining, route, cur = list(points), [], start
54     while remaining:
55         k = int(np.argmin([float(np.linalg.norm(p - cur)) for p in
56                             remaining]))
57         route.append(remaining.pop(k)); cur = route[-1]
58         improved = True
59         while improved:
60             improved = False
61             for i in range(len(route) - 2):
62                 for j in range(i + 2, len(route)):
63                     a = start if i == 0 else route[i - 1]
64                     b, c = route[i], route[j]
65                     d = route[j + 1] if j + 1 < len(route) else None
66                     old = float(np.linalg.norm(a - b)) +
67                         (float(np.linalg.norm(c - d)) if d is not None else 0.0)
68                     new = float(np.linalg.norm(a - c)) +
69                         (float(np.linalg.norm(b - d)) if d is not None else 0.0)
70                     if new + 1e-9 < old:
71                         route[i:j + 1] = reversed(route[i:j + 1]); improved = True
72     return route
73
74 def wedge_halfplanes(S: np.ndarray, theta: float, eps:
75     float = EPS_DEG):
76     """示向度带 [theta-eps, theta+eps] 转两个半平面 (a,b,c):
77         a*x+b*y+c>=0。"""
78     u, v = unit(theta - eps), unit(theta + eps)
79     n1 = np.array([-u[1], u[0]]); h1 = (n1[0], n1[1], -float(n1
80         @ S))
81     n2 = np.array([v[1], -v[0]]); h2 = (n2[0], n2[1], -float(n2
82         @ S))
83     return [h1, h2]
84
85 def clip_halfplane(poly: list, hp) -> list:
86     """Sutherland - Hodgman: 用单个半平面裁剪凸多边形。"""
87     a, b, c = hp

```

```

83     if not poly:
84         return []
85     out, n = [], len(poly)
86     for i in range(n):
87         p, q = poly[i], poly[(i + 1) % n]
88         pin = a * p[0] + b * p[1] + c >= -1e-9
89         qin = a * q[0] + b * q[1] + c >= -1e-9
90         if pin:
91             out.append(p)
92         if pin != qin:
93             den = a * (q[0] - p[0]) + b * (q[1] - p[1])
94             if abs(den) > 1e-12:
95                 t = -(a * p[0] + b * p[1] + c) / den
96                 out.append(p + t * (q - p))
97     return out
98
99
100 def locate_region(points, bearings, eps: float = EPS_DEG,
101                   box_half=2e5):
102     """交会定位区域 =
103         所有示向度带（半平面）的交集凸多边形。返回 (poly,
104         unbounded)，空则 poly=None。"""
105     if len(points) != len(bearings) or len(points) < 2:
106         raise ValueError("需要至少2个检测点且数量一致")
107     ctr = np.mean([np.asarray(p, dtype=float) for p in points],
108                   axis=0)
109     L = box_half
110     poly = [ctr + np.array([-L, -L]), ctr + np.array([L, -L]),
111             ctr + np.array([L, L]), ctr + np.array([-L, L])]
112     for S, th in zip(points, bearings):
113         for hp in wedge_halfplanes(np.asarray(S, dtype=float),
114                                   float(th), eps):
115             poly = clip_halfplane(poly, hp)
116         if not poly:
117             return None, False
118     unbounded = max(float(np.abs(v).max()) for v in poly) > 1e4
119     return poly, unbounded
120
121
122 def welzl_min_enclosing_circle(points) -> Tuple[np.ndarray,

```

```

    float]:
118     """Welzl 随机增量法求点集最小外接圆, 返回 (center,
        radius)。"""
119     pts = [np.asarray(p, dtype=float) for p in points]
120     n = len(pts)
121     if n == 0:
122         return np.zeros(2), 0.0
123     if n == 1:
124         return pts[0].copy(), 0.0
125
126     def c2(a, b):
127         return (a + b) / 2.0, float(np.linalg.norm(a - b) / 2.0)
128
129     def c3(a, b, c):
130         ax, ay = a; bx, by = b; cx, cy = c
131         d = 2.0 * (ax * (by - cy) + bx * (cy - ay) + cx * (ay - by))
132         if abs(d) < 1e-12:
133             pairs = [(a, b), (b, c), (a, c)]
134             return max((c2(x, y) for x, y in pairs), key=lambda t: t[1])
135         ux = ((ax**2 + ay**2) * (by - cy) + (bx**2 + by**2) * (cy -
            ay) + (cx**2 + cy**2) * (ay - by)) / d
136         uy = ((ax**2 + ay**2) * (cx - bx) + (bx**2 + by**2) * (ax -
            cx) + (cx**2 + cy**2) * (bx - ax)) / d
137         cc = np.array([ux, uy])
138         return cc, float(np.linalg.norm(a - cc))
139
140     def welzl(P, R, k):
141         if k == 0 or len(R) == 3:
142             if len(R) == 0: return np.zeros(2), 0.0
143             if len(R) == 1: return R[0].copy(), 0.0
144             if len(R) == 2: return c2(R[0], R[1])
145             return c3(R[0], R[1], R[2])
146         p = P[k - 1]
147         cc, r = welzl(P, R, k - 1)
148         if float(np.linalg.norm(p - cc)) <= r + 1e-9:
149             return cc, r
150         return welzl(P, R + [p], k - 1)
151
152     perm = np.random.default_rng(42).permutation(n)
153     return welzl([pts[i] for i in perm], [], n)

```

```

154
155
156 #
=====
157 # 2. 迭代定位 / 射线交段 (位置估计用)
158 #
=====
159 def ray_circle_interval(s: np.ndarray, u: np.ndarray,
    radius: float = TARGET_RADIUS):
160     """射线 s+t*u (t 0) 与目标圆盘的参数区间
        [t_lo,t_hi], 无交集返回None。"""
161     B = 2.0 * float(s @ u)
162     C = float(s @ s - radius ** 2)
163     disc = B * B - 4.0 * C
164     if disc < 0:
165         return None
166     root = math.sqrt(max(0.0, disc))
167     t1, t2 = (-B - root) / 2.0, (-B + root) / 2.0
168     lo, hi = max(0.0, min(t1, t2)), max(t1, t2)
169     return (lo, hi) if hi >= lo else None
170
171
172 def iterative_localize(observations,
    clear_r=CLEAR_RADIUS_MARGIN):
173     """由若干 (检测点, 示向度) 观测求定位区域及其最小外接圆。
174     返回 (poly, center, radius, can_clear, unbounded)。"""
175     stations = [o[0] for o in observations]
176     bearings = [o[1] for o in observations]
177     poly, unb = locate_region(stations, bearings)
178     if poly is None or len(poly) < 3 or unb:
179         return poly, None, float("inf"), False, unb
180     center, radius = welzl_min_enclosing_circle(np.array(poly))
181     return poly, center, radius, radius <= clear_r, unb
182
183
184 #
=====
185 # 3. 频道状态
186 #
=====

```

```

187 @dataclass
188 class ChannelState:
189     channel: int
190     status: str = "unknown"          # unknown / detected /
        near / cleared
191     observations: List[Tuple[Tuple[float, float], float]] =
        field(default_factory=list)
192     no_signal_positions: List[Tuple[float, float]] =
        field(default_factory=list)
193     tried_keys: set = field(default_factory=set)    #
        该频道已成功执行measure的量化坐标，禁止重复
194     clear_attempts: int = 0
195
196
197     #
        =====
198     # 4. 模拟器 HTTP 客户端（严格符合附件2）
199     #
        =====
200
201 class RobotClient:
202     def __init__(self, base_url=BASE_URL, robot_id=ROBOT_ID):
203         self.base_url, self.robot_id = base_url, robot_id
204         self.counter = 0
205         self.log: List[Dict[str, Any]] = []
206
207     def _next_id(self, prefix):
208         self.counter += 1
209         return f"{prefix}-{self.counter}"
210
211     def _post(self, path, payload):
212         """POST: 新动作新 request_id; 网络断连在重试中复用同一
213             payload (含同一 request_id)。"""
214         data = json.dumps(payload).encode("utf-8")
215         last = None
216         for attempt in range(MAX_RETRY):
217             try:
218                 req = Request(self.base_url + path, data=data,
219                             headers={"Content-Type": "application/json"}, method="POST")
220                 with urlopen(req, timeout=HTTP_TIMEOUT) as resp:
221                     result = json.loads(resp.read().decode("utf-8"))

```

```

220     self.log.append({"path": path, "request": payload,
221                    "response": result, "time": time.time()})
222     return result
223 except HTTPError as e:
224     print(f"[HTTP ERROR] {path} -> {e.code} {e.reason}")
225     return None
226 except (URLError, TimeoutError, ConnectionError, OSError)
227         as e:
228     last = e
229     if attempt < MAX_RETRY - 1:
230         w = RETRY_BACKOFF * 2 ** attempt
231         print(f"[网络重试] {path} 第{attempt+1}次失败:
232               {e}, {w:.1f}s后重试")
233         time.sleep(w)
234     except json.JSONDecodeError:
235         return None
236     print(f"[ERROR] {path} 重试{MAX_RETRY}次仍失败: {last}")
237     return None
238
239 def enter(self):
240     return self._post("/enter", {"arena_id": ARENA_ID,
241                                "robot_id": self.robot_id,
242                                "request_id": self._next_id("enter")})
243
244 def exit(self):
245     return self._post("/exit", {"arena_id": ARENA_ID,
246                                "robot_id": self.robot_id,
247                                "request_id": self._next_id("exit")})
248
249 def measure(self, pos, channel):
250     return self._post("/measure", {
251         "arena_id": ARENA_ID, "robot_id": self.robot_id,
252         "request_id": self._next_id("measure"),
253         "position": {"x": float(pos[0]), "y": float(pos[1])},
254         "channel": int(channel)})
255
256 def clear(self, pos, channel):
257     return self._post("/clear", {
258         "arena_id": ARENA_ID, "robot_id": self.robot_id,
259         "request_id": self._next_id("clear"),

```

```

254         "position": {"x": float(pos[0]), "y": float(pos[1])},
255         "channel": int(channel)})
256
257     #
258     =====
259     # 5. 问题四求解器 (全向 + 180°定向混合)
260     #
261     =====
262
263     class Q4Solver:
264     def __init__(self, client):
265         self.client = client
266         self.discovery_pts = [np.array(p, dtype=float) for p in
267             Q4_DISCOVERY_POINTS]
268         self.channels: Dict[int, ChannelState] = {c:
269             ChannelState(c) for c in range(1, 21)}
270         self.cleared_count = 0
271         self.real_start = 0.0
272         self.real_budget = 1200.0
273         self.virtual_time = 0.0
274         self.current_pos = np.zeros(2)
275         self.inplace_failed = set() #
276         发现网途中就地清除未中、交回追踪补测的频道
277
278     # ----- 基础 -----
279     def time_ok(self, reserve=30.0) -> bool:
280         if time.time() - self.real_start > self.real_budget -
281             reserve:
282         return False
283         return self.virtual_time <= VIRTUAL_LIMIT
284
285     def _pkey(self, pos):
286         return (round(float(pos[0]) / POS_GRID),
287             round(float(pos[1]) / POS_GRID))
288
289     def _is_tried(self, ch, pos) -> bool:
290         return self._pkey(pos) in self.channels[ch].tried_keys
291
292     def _first_unused(self, ch, cand, legal=None):
293         """按候选顺序返回第一个没测过且合法的点；都测过返回None。"""

```

```

286     for p in candb:
287         p = np.asarray(p, dtype=float)
288         if not np.all(np.isfinite(p)) or self._is_tried(ch, p):
289             continue
290         if legal is not None and not legal(p):
291             continue
292         return p
293     return None
294
295     @staticmethod
296     def _legal_q4_point(p, limit=2800.0):
297         #
298         定向源可能贴边界且朝外辐射，追踪允许到圆外（与发现网外扩一致），但不超2
299
300         return np.all(np.isfinite(p)) and float(np.linalg.norm(p))
301             <= limit
302
303     def _measure(self, pos, ch):
304         resp = self.client.measure(pos, ch)
305         if resp is None:
306             return None
307         if resp.get("accepted") is not True:
308             return "rejected"
309         self.virtual_time = float(resp.get("virtual_time_s",
310             self.virtual_time))
311         self.current_pos = np.asarray(pos, dtype=float)
312         return resp.get("measure_result")
313
314     def _clear(self, pos, ch):
315         resp = self.client.clear(pos, ch)
316         if resp is None:
317             return None
318         if resp.get("accepted") is not True:
319             return "rejected"
320         self.virtual_time = float(resp.get("virtual_time_s",
321             self.virtual_time))
322         self.current_pos = np.asarray(pos, dtype=float)
323         return resp.get("clear_result")
324
325     def _svd(self):
326         return self.client.log[-1]["response"].get("svd_deg") if

```

```

        self.client.log else None

322
323 def _absorb(self, ch, pos, result, tag=""):
324     """统一吸收一次测量结果（登记已测坐标；Q4的no_signal只记录、不做距离排除）
325     st = self.channels[ch]
326     gp = (float(pos[0]), float(pos[1]))
327     st.tried_keys.add(self._pkey(gp))
328     if result == "direction":
329         svd = self._svd()
330         if svd is not None:
331             st.observations.append((gp, float(svd)))
332             st.status = "detected"
333         elif result == "near":
334             st.status = "near"
335         if self._clear(gp, ch) == "success":
336             st.status = "cleared"; self.cleared_count += 1
337         print(f"    频道{ch} near→清除成功")
338         elif result == "no_signal":
339             st.no_signal_positions.append(gp)
340         return st.status
341
342     # ----- 阶段1：27点完备发现网巡回 -----
343     def _clear_ready_in_place(self):
344         """发现网某点扫完后：就近清除已交会收敛(MEC 19m)且距当前 700m的源。
345         只做 clear
346         这一早晚要发生的终态动作以省折返，绝不提前补点打乱巡回节奏；
347         清除未中登记交回追踪，保证不漏。"""
348         while self.time_ok():
349             best = None
350             for ch, st in self.channels.items():
351                 if st.status != "detected" or len(st.observations) < 2 or
352                     ch in self.inplace_failed:
353                     continue
354                 _, center, radius, can_clear, unb =
355                     iterative_localize(st.observations)
356                 if can_clear and center is not None and not unb:
357                     d = float(np.linalg.norm(center - self.current_pos))
358                     if d <= INPLACE_CLEAR_MAXD and (best is None or d <
359                         best[0]):
360                         best = (d, ch, center, radius)

```

```

357     if best is None:
358         return
359     _, ch, center, radius = best
360     st = self.channels[ch]
361     st.clear_attempts += 1
362     if self._clear(center, ch) == "success":
363         st.status = "cleared"; self.cleared_count += 1
364         print(f"    频道{ch}
365             发现网途中就近清除（定位半径{radius:.1f}m）")
366     if self.cleared_count >= MAX_SOURCES:
367         return
368     else:
369         self.inplace_failed.add(ch)
370         return
371
372     def discovery_tour(self):
373         """27点完备发现网按2-opt最短巡回，逐点扫所有未清除频道。
374         已detected也继续测以免费攒交会；每点后就近清除收敛源、就地侧移补单观测源。
375         print("[阶段1]
376             Q4完备发现网巡回（27点，任意朝向必被发现，途中就近清除）...")
377         route = two_opt_route(self.current_pos,
378                               list(self.discovery_pts))
379         for gp_arr in route:
380             if not self.time_ok() or self.cleared_count >= MAX_SOURCES:
381                 return
382             gp = (float(gp_arr[0]), float(gp_arr[1]))
383             for ch in range(1, 21):
384                 if self.channels[ch].status == "cleared":
385                     continue
386                 if not self.time_ok():
387                     return
388                 self._absorb(ch, gp, self._measure(gp, ch))
389                 self._clear_ready_in_place() # 先清已收敛源
390                 self._inplace_approach(gp) #
391                 再给本点单观测源±200侧移补第2观测
392             if self.cleared_count >= MAX_SOURCES:
393                 return
394             found = [c for c, s in self.channels.items() if s.status in
395                     ("detected", "near")]
396             print(f"    发现网巡回完成：发现{len(found)}个 {found}")

```

```

392
393 def _inplace_approach(self, gp):
394     """发现网途中：对'唯一观测就在当前点gp'的源，就地用最小侧移(±200m)补第2观测
395     机器狗此刻就在首见点，侧移仅200m(40s)；补到即交会清除，避免追踪阶段长距离补
396     ±200都no_signal则不纠缠、留给追踪阶段。"""
397     gp_arr = np.asarray(gp, float)
398     for ch, st in self.channels.items():
399         if st.status != "detected" or len(st.observations) != 1:
400             continue
401         anchor = np.asarray(st.observations[0][0], float)
402         if float(np.linalg.norm(anchor - gp_arr)) > 1e-6:
403             continue #
404             唯一观测不在本点，不处理
405         svd = st.observations[0][1]
406         u = unit(svd); n = np.array([-u[1], u[0]])
407         for cand in (anchor + 200.0 * n, anchor - 200.0 * n):
408             if self.cleared_count >= MAX_SOURCES or not self.time_ok():
409                 return
410             p = self._first_unused(ch, [cand], self._legal_q4_point)
411             if p is None:
412                 continue
413             self._absorb(ch, p, self._measure(p, ch))
414             if st.status == "cleared" or len(st.observations) >= 2:
415                 break #
416                 补到第2条即停，交会清除统一处理
417             self._clear_ready_in_place()
418
419 # ----- 阶段2：定向源专用追踪 -----
420 def _approach_candidates(self, st):
421     """定向源只有1条示向时的补点序列（首见点 S
422     已在其180°覆盖半平面内）。
423     沿示向 u 大步直走、当 S
424     离源较近时会一步穿过源落到覆盖半平面正对面而 no_signal,
425     故分两阶段、按序枚举，由 process 用 tried_keys
426     逐个试、no_signal 即换、绝不重复：
427     阶段A 纯侧移（不沿u前进，不会穿过源）：左右交错、由小到大；
428     阶段B
429     先沿u逼近再侧移（中远源/贴接收半径边界），逼近缩短距离后再侧移。"""
430     pos, svd = st.observations[-1]
431     s = np.asarray(pos, float); u = unit(svd); n =

```

```

    np.array([-u[1], u[0]])
426 cand_s = []
427 for off in (200.0, 400.0, 600.0): #
    阶段A: 以S为锚的纯侧移, 左右交错
428 cand_s += [s + off * n, s - off * n]
429 for d, off in ((200.0, 300.0), (400.0, 350.0), (600.0,
    400.0)): # 阶段B: 逼近后侧移
430 cand_s += [s + d * u + off * n, s + d * u - off * n]
431 cand_s += [s + 150.0 * u] #
    纯逼近兜底 (同方位、距离更近、仍在覆盖内)
432 return cand_s
433
434 def _third_candidates(self, st, center):
435 """ 2条示向但区域未收敛: 朝估计中心逼近并在其四周小环补交会。
436 离源越近越不易穿出180°覆盖半平面, 比大跨度法向横移更稳。"""
437 cur = self.current_pos
438 cand_s = []
439 to = np.asarray(center, float) - cur
440 d = float(np.linalg.norm(to))
441 if d > 260.0:
442 cand_s.append(cur + to / d * (d - 200.0)) #
    朝源走到距估计中心约200m
443 for a in range(0, 360, 60): #
    中心四周250m小环补方向
444 cand_s.append(np.asarray(center, float) + 250.0 *
    unit(float(a)))
445 cand_s += self._approach_candidates(st) #
    兜底: 退回沿示向逼近序列
446 cand_s.sort(key=lambda p: float(np.linalg.norm(p - cur)))
447 return cand_s
448
449 def _ray_midpoint(self, S1, theta):
450 """ 单次观测: 源在示向射线与目标圆交段上, 取该段中点作位置估计 (用于最近邻估计)
451 S1 = np.asarray(S1, dtype=float); u = unit(theta)
452 iv = ray_circle_interval(S1, u, TARGET_RADIUS)
453 if iv is None:
454 return S1 + 900.0 * u
455 t0 = max(NEAR_THRESHOLD, iv[0]); t1 =
    min(MAX_RECEIVE_RADIUS, iv[1])
456 t = 0.5 * (t0 + t1) if t1 > t0 else 900.0

```

```

457     return S1 + t * u
458
459     def _est_pos(self, ch):
460         """待清源的估计位置（用于最近邻调度）： 2次观测取最小外接圆圆心，否则取射线
461         st = self.channels[ch]
462         if len(st.observations) >= 2:
463             poly, unb = locate_region([o[0] for o in st.observations],
464                                       [o[1] for o in st.observations])
465             if poly is not None and len(poly) >= 3 and not unb:
466                 c, _ = welzl_min_enclosing_circle(np.array(poly))
467             return c
468         if st.observations:
469             return self._ray_midpoint(*st.observations[0])
470         return np.zeros(2)
471
472     def process_channel(self, ch):
473         """单个（含定向）源的定位清除。no_signal在Q4三义（无源/超距/在背面），不做
474         也绝不重测同一点（tried_keys兜底），立即换下一候选方向。
475         返回 cleared / progress（观测有增长可续试）/
476             stalled（零增长、候选耗尽，重选也无新信息）。"""
477         st = self.channels[ch]
478         n0 = len(st.observations)
479         used = 0
480         while used < Q4_MEASURE_CAP:
481             if not self.time_ok():
482                 break
483             if len(st.observations) == 1:
484                 # 仅1观测：侧移/逼近补第2条
485                 npt = self._first_unused(ch, self._approach_candidates(st),
486                                         self._legal_q4_point)
487             else:
488                 poly, center, radius, can_clear, unb =
489                     iterative_localize(st.observations)
490                 if poly is not None and len(poly) >= 3 and not unb and
491                     can_clear:
492                     st.clear_attempts += 1
493                     if self._clear(center, ch) == "success":
494                         st.status = "cleared"; self.cleared_count += 1
495                     print(f"    频道{ch} 清除成功（定位半径{radius:.1f}m）");
496                     return "cleared"

```

```

491     npt = self._first_unused(ch, self._third_candidates(st,
492         center), self._legal_q4_point)
493     else:
494         #
495         交会退化：继续补观测
496     npt = self._first_unused(ch, self._approach_candidates(st),
497         self._legal_q4_point)
498     if npt is None:
499         break
500     self._absorb(ch, npt, self._measure(npt, ch)); used += 1
501     if st.status == "cleared":
502         return "cleared"
503     if len(st.observations) >= 2:
504         #
505         兜底：用最新估计最后清一次
506     _, center, _, _, _ = iterative_localize(st.observations)
507     if center is not None and self._clear(center, ch) ==
508         "success":
509         st.status = "cleared"; self.cleared_count += 1
510         print(f"    频道{ch} 最后尝试清除成功"); return "cleared"
511         progressed = len(st.observations) > n0
512         if not progressed:
513             print(f"    频道{ch}
514                 本轮无新观测（观测{len(st.observations)}次），挂起待全局救援")
515         else:
516             print(f"    频道{ch}
517                 未能清除（观测{len(st.observations)}次，有进展待续）")
518         return "progress" if progressed else "stalled"
519
520     def _rescue(self, stalled):
521         """全局救援：对常规补点序列仍只拿到 1 条观测的刁钻定向源，以其首见点为圆心做
522         小半径多方向密环。首见点既然可见，其足够小邻域内必有仍落在180°覆盖半平面、
523         变化很小的点，以此强行补到第2条观测再交会收敛。"""
524         if not stalled:
525             return
526         print(f"[阶段2b] 对{len(stalled)}个卡住源做近邻密环救援
527             {sorted(stalled)}")
528         for ch in list(stalled):
529             st = self.channels[ch]
530             if st.status == "cleared" or not st.observations:
531                 continue
532             if not self.time_ok():

```

```

523     return
524     anchor = np.asarray(st.observations[-1][0], float)
525     before = len(st.observations)
526     for r in (100.0, 180.0, 280.0):
527         if st.status == "cleared":
528             break
529         for a in range(0, 360, 30):
530             if not self.time_ok():
531                 return
532             p = anchor + r * unit(float(a))
533             p = self._first_unused(ch, [p], self._legal_q4_point)
534             if p is None:
535                 continue
536             self._absorb(ch, p, self._measure(p, ch))
537             if st.status == "cleared":
538                 break
539             if len(st.observations) >= before + 1 and
                len(st.observations) >= 2:
540                 self.process_channel(ch)                #
                    拿到新观测，交常规流程收敛
541             break
542
543     def track(self):
544         """阶段2：发现网走完后所有源必已被发现，按最近邻逐个定位清除。
545         stalled 集合保证观测零增长的源不被无限重选；progress
            允许续试，候选受 tried_keys
546         限制终将收敛或转 stalled；另设全局轮数硬上限双保险，最后对
            stalled 做一次密环救援。"""
547         print("[阶段2] Q4逐源追踪清除（侧移补观测 + 交会）...")
548         stalled = set()
549         rounds, hard_cap = 0, 20 * Q4_MEASURE_CAP
550         while self.time_ok() and self.cleared_count < MAX_SOURCES
            and rounds < hard_cap:
551             pending = [c for c, s in self.channels.items()
552                 if s.status in ("detected", "near") and c not in stalled]
553             if not pending:
554                 break
555             pending.sort(key=lambda c:
                float(np.linalg.norm(self._est_pos(c) -
                    self.current_pos)))

```

```

556     ch = pending[0]; rounds += 1
557     print(f"
        处理频道{ch}（已有{len(self.channels[ch].observations)}次观测）...")
558     if self.process_channel(ch) == "stalled":
559         stalled.add(ch)
560         self._rescue(stalled)
561
562     # ----- 主流程 -----
563     def run(self, save_log="robot_log_q4.json"):
564         enter = self.client.enter()
565         if enter is None or enter.get("accepted") is not True:
566             print("[ERROR] /enter
                失败（确认模拟器已进入问题四测试并过5秒倒计时）"); return
567         self.real_budget =
            float(enter.get("remaining_real_duration_s", 1200.0))
568         self.real_start = time.time()
569         print(f"[INFO] 问题四
            可用现实时间={self.real_budget:.0f}s")
570         self.discovery_tour() #
            阶段1: 27点完备发现网巡回
571         self.track() #
            阶段2: 定向源专用追踪（含密环救援）
572         print(f"[INFO] 任务结束，共清除 {self.cleared_count}
            个干扰源，虚拟时间 {self.virtual_time:.0f}s")
573         self.client.exit()
574         if save_log:
575             with open(save_log, "w", encoding="utf-8") as f:
576                 json.dump(self.client.log, f, ensure_ascii=False, indent=2)
577         print(f"[INFO] 行为日志已保存到
            {save_log}（{len(self.client.log)}条）")
578
579     #
        =====
580
581     # 6. 入口
582     #
        =====
583
584     def main():
        ap = argparse.ArgumentParser(description="CUMCM-B 问题四
            全向+180°定向混合 自动定位清除（独立正式版）")

```